

Fable 5 重新想象可能性：从长程委托、验证 闭环到动态 workflows

SSS & Codex

2026 年 7 月 1 日



视频作者/频道: Every

发布日期: 2026-06-10

视频时长: 52:06

视频链接: <https://www.youtube.com/watch?v=XWpTgCvgYaE>

PDF 制作 Skill: <https://github.com/wdkns/wdkns-skills>

目录

1 导言：为什么 Fable 5 迫使人重新想象可能性	3
1.1 先给出本节结论	3
1.2 发布当天的兴奋容易误导	3
1.3 新技能缺口：会编排代理的人先看见变化	3
1.4 本章小结	4
2 第一章：从提示词到委托	4
2.1 旧的交互模型开始显得短	4
2.2 一夜之间推进任务：长时间尺度的含义	5
2.3 验证是委托的验收边界	6
2.4 本章小结	6
3 第二章：Sonnet、Opus 与 Fable	6
3.1 模型选择是一道资源分配题	6
3.2 前置规划、文档和团队对齐	7
3.3 subagents 与主线程响应性	7
3.4 effort levels、Sonnet 与 Opus 的谨慎比较	8
3.5 本章小结	9
4 第三章：周末媒体追踪器：agent-native 架构的最小样本	9
4.1 周末上下文：为什么这个例子有教学价值	9
4.2 入口降低：终端、桌面 app 与非工程师用户	9
4.3 个人媒体追踪需求：从推荐碎片到可维护清单	10
4.4 把软件团队靠近软件：agent-native 的最小结构	11
4.5 编辑请求与实时预览：产品自己长出修改回路	12
4.6 为什么这是最小样本：生产化能力还要补课	14
4.7 本章小结	14
5 第四章：成本坍缩：从 Instagram v1 到个人工具	14
5.1 从 Instagram v1 看见时间尺度的变化	15
5.2 想法到产物：真正下降的是任务级成本	15
5.3 协调损耗：成本公式里的关键分量	16
5.4 内部招聘工具：协调成本下降的证据	17
5.5 成本坍缩的风险：可建造性提高，判断压力也提高	17
5.6 本章小结	17
6 第五章：软件工程结束了吗？职责边界正在重画	18
6.1 旧定义：工程师把复杂性压进编辑器	18
6.2 职责链条：信任半径扩大，角色边界重画	18
6.3 风险链条：错误会走得更远	19
6.4 人类仍负责 taste、judgment 和 accountability	19
6.5 情感维度：失落与扩展产出同时存在	20
6.6 本章小结	20

7 第六章：Anthropic 团队今天怎样协作	20
7.1 DRI：代理并行之后，人类责任更清楚	21
7.2 许多 Claude 会话需要新的工作面板	22
7.3 生产意识：代码写完以后，工程才刚进入风险区	23
7.4 原型主导对齐：让想法先变成可讨论的东西	24
7.5 成本治理：有效使用比用量排行榜更重要	24
7.6 代码审查：代理开始拥有“判断”，人类仍要校准	25
7.7 接口变化：移动端、远程开发盒和执行地点分离	25
7.8 把模型上下文转回人脑	26
7.9 多人协作：从分享聊天到共享工作状态	27
7.10 合并前问责：能上线之前，必须能解释	28
7.11 本章小结	28
8 第七章：验证机制：让代理做完，也让人敢签字	28
8.1 从真实流程开始：staging、真实账号与跳过无关 onboarding	29
8.2 两条路径一起跑：well-known paths 与 current-change paths	29
8.3 视觉证据：screenshots、screen shares 与 video captures	30
8.4 复杂系统的替身：mock backend 与 in-memory substitutes	31
8.5 反馈闭环：Slack/MCP、PR、部署与客户跟进	31
8.6 修复判断：立即止血与长期治理要分层	32
8.7 给团队的验证清单	32
8.8 本章小结	33
9 第八章：还能建什么：领域专家、MCP 与复杂系统	33
9.1 从 3D 游戏到复杂仿真：表达领域复杂性	34
9.2 MCP、权限和部署：内部系统才是真正的复杂系统	35
9.3 Senior engineer benchmark：有用，也要带着限定读	35
9.4 本章小结	35
10 第九章：动态工作流：把长程任务写成可运行的流程	36
10.1 迁移案例：Python 到 TypeScript/Bun	36
10.2 为什么代码化表达很重要	37
10.3 模型选择也会进入工作流内部	38
10.4 本章小结	38
11 总结与延伸：新能力、新约束、新工程常识	38
11.1 新能力：把复杂想法做成可运行系统	39
11.2 新约束：权限、验证、成本和责任	39
11.3 新工程常识：把工作写成流程	39
11.4 本章小结	39

1 导言：为什么 Fable 5 迫使人重新想象可能性

1.1 先给出本节结论

这一集开场要解决的核心问题很直接：Fable 5 的价值应放在连续几周的真实使用里判断，看它怎样改变工作分配、验证责任和人的技能结构。Dan 在 ‘00:00:21–00:00:43’ 把采访目标放在 *beyond the first day*：他想知道这种模型在日常生活和工作中究竟改了什么、保留了什么。Mike 随后在 ‘00:00:59–00:01:21’ 回应，发布首日印象太浅，真正的理解来自持续使用；他还用早先 Opus 系列的经验说明，人们常常要花更长时间才意识到自己可以“推得更远”。

全书主线

Fable 5 迫使人重新想象“可能性”，关键原因在于任务时间尺度变长：人的工作从写一个提示词，扩展为定义意图、装载上下文、安排多个代理、审查证据，并承担最终判断。

更准确的说法是：模型的可委托任务长度变长，导致人的瓶颈从“如何让模型答对一句话”转移到“如何让一组任务在可验证的边界内推进”。类比一下，早期计算器让人少算几步；这里更像把一个会写代码、会看上下文、会遇阻绕行的初级工程小组接进工作流。人依然负责目标、品味、风险接受和最终签字。

1.2 发布当天的兴奋容易误导

Dan 在 ‘00:00:25–00:00:43’ 明确担心一种“AI psychosis”式的过热反应：看到一个强模型发布，马上把所有能力想象成已经稳定进入自己的生活。Mike 的回答给出一个更硬的判断标准：连续几周使用之后，人才会发现自己起初没有把任务推到足够远的地方（‘00:01:03–00:01:21’）。这里的“远”包含三个层次：

- 时间更远：任务可以跨过几小时、一个夜晚，甚至等待外部服务恢复。
- 上下文更远：模型需要理解功能演进、团队约定和系统结构，当前一句话只是入口。
- 责任更远：完成输出之后还要回到人的审查、测试和验收。

因此，本 PDF 后续章节会把 Fable 5 当成一种“委托系统”来讲。委托系统的价值来自可持续的任务推进；它的风险也来自同一个地方，因为越长的任务链越容易累积错误、误解和未验证假设。

1.3 新技能缺口：会编排代理的人先看见变化

Dan 在 ‘00:01:23–00:01:44’ 提到一个分化现象：非技术知识工作者可能还在问“我能拿它做什么”，而已经在 *orchestrating agents* 的人会感到自己需要学习一批新技能。这里的“代理编排”可以理解为使多个半自主执行者围绕同一目标分工：有人做规划，有人写代码，有人抽查结果，有人生成证据。普通的提问技巧只能覆盖其中一小段。

什么叫代理编排

有效的代理编排会把任务拆成角色、边界、输入、输出和验收条件：例如一个代理负责写实现，一个代理负责查证事实，一个代理负责跑测试，一个代理负责指出遗漏。重复发送同一个请求只会放大噪声。真正的难点在于让它们共享足够上下文，同时避免互相覆盖和重复劳动。

这里有一个读者容易漏掉的“未知未知”：能力上升以后，人的表达负担也会上升。任务越大，提示词越像项目 brief；上下文越长，错误前提越容易污染整条链；代理越多，协调成本越接近小团队管理。Fable 5 的强处会奖励这类组织能力，也会惩罚含混目标和缺失验收。

对话要点摘要 (00:01:39–00:01:44)

Dan: 那些已经在编排代理的人，会突然觉得自己还有很多新东西要学。

这句短对话的教学价值在于，它把模型升级带来的变化从“工具升级”拉回到“工作技能升级”。会写提示词只是起点；会设定任务边界、上下文、权限、验证和交接，才是长程代理时代的核心能力。

1.4 本章小结

本节的核心 claim 是：Fable 5 的真正变化要从持续使用中观察，而发布首日只能看到表层震撼。机制上，模型把可委托任务的时间跨度、上下文跨度和责任跨度同时拉长。实践上，构建者和团队需要把“会问问题”升级为“会组织代理、会设置证据、会承担验收”。

2 第一章：从提示词到委托

2.1 旧的交互模型开始显得短

Mike 在 ‘00:01:48–00:02:28’ 描述自己的第一反应：他从 CPO 回到 Labs，进入 *builder mode* 后，突然觉得自己原来的 prompting 方式和任务拆解方式已经过时。这里的重点在于交互模型变短；某个提示词模板失效只是表层现象。过去常见流程是：人给一个小任务，模型回一个答案；人再追问；模型再修改。Fable 5 让人开始把“我要什么、为什么、后面会如何演进、系统里已有约束是什么”一次性装进去。

Mike 在 ‘00:02:49–00:03:01’ 提到，模型不只在 one-shot 输出上令人印象深刻，还能理解意图的后续演化和 *global context*。可以把这看成 workflow 里的坐标系变化：早期聊天像在单个坐标点上问路，长程委托像把地图、目的地、交通规则和中途检查点都交给一个执行者。

从提示词到委托

提示词关注“这一轮怎样回答”；委托关注“这个目标怎样被推进到可验收状态”。委托必须包含意图、上下文、边界、可接受的绕行策略、证据要求和人工复核点。

From prompt loop to delegation

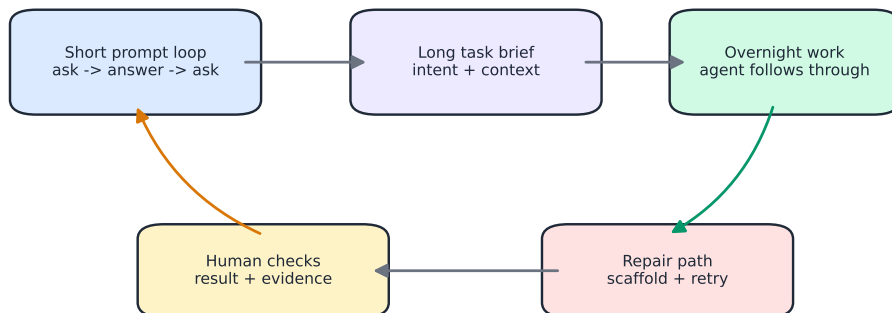


图 1: 从短轮次提示词循环到长程委托: 人的输入从单轮问题扩展为意图、上下文、边界、证据要求和修复路径。此图根据 ‘00:01:48–00:04:13’ 中关于 builder mode、global context、overnight delegation 和 scaffolded backend 的讲解重绘。

2.2 一夜之间推进任务：长时间尺度的含义

最有教学价值的例子出现在 ‘00:03:10–00:04:13’。Mike 说自己在坐飞机、网络可能中断时，也可以先设好上下文和指令；最近两个月里，他多次把 Claude 放到复杂任务上，然后 *wish Claude a good night*。第二天醒来，任务通常已经在凌晨完成。这个例子说明，人的工作从“盯着模型每一步”变成“先把任务环境搭好，再让代理独立跑一段”。

真正关键的细节在后半段：远端服务宕掉时，模型没有直接停在阻塞处。Mike 描述它会写一个临时的 scaffolded backend，记录这个临时处理的限制，继续把任务推进，等服务恢复后再修正（‘00:03:53–00:04:13’）。这就是 *complete the swing* 的含义：一杆打出去之后，动作要完整收住；遇到中途阻塞时，系统应该找合理临时支架、留下审计线索，并把可推进的部分推进到验收面前。

scaffolded backend 的直觉

scaffolded backend 可以理解为“临时后台支架”。例如真实远端 API 暂时不可用，代理先写一个假服务或本地替身，让前端、数据流和测试路径继续跑起来，同时记录真实服务恢复后需要替换的位置。它像施工时的脚手架：不会成为最终建筑主体，却能让工程继续推进。

对话要点摘要（00:03:31–00:04:26）

Mike: 他会把 Claude 安排到复杂任务上，晚上说一句 *wish Claude a good night*；遇到远端服务宕掉时，Claude 会先搭临时 backend、记录限制，并继续推进。结果回来之后，仍然要 review 和 verification。

这段对话值得保留，因为它把“代理能干活”讲成了一个完整工作场景：任务过夜、服务故障、临时替身、文档记录、后续修复、人工验收。这里没有神话化自动化；它把强模型放回工程现实里。

2.3 验证是委托的验收边界

Mike 在 ‘00:04:20–00:04:30’ 明确补上限制：结果需要人工 review，还存在一个完整的 verification 问题，并且验证本身也是完成动作的一部分。这个 caveat 很重要，因为长程代理最危险的失败方式常常表现为“看起来做完，关键证据缺失”。如果代理搭了临时 backend，却没有记录它是临时支架，后续维护者会把支架误当成正式设施；如果代理跑了几条顺路测试，却没有覆盖真实用户路径，交付风险会被低估。

把“完成”拆成两个状态

代理说任务完成，只代表它到达了某个输出状态。团队可以接受任务完成，需要额外看到证据状态：哪些真实流程跑过，哪些替代件仍在用，哪些假设等待确认，哪些风险由人签字接受。

这个章节的隐含工程常识是：委托越强，验收越要硬。没有验证的长程执行会把错误埋得更深；有验证的长程执行才会释放人类时间。后续第七章会专门展开验证机制，这里先记住一个原则：Fable 5 改变的是可委托范围，同时提高了验收设计的重要性。

2.4 本章小结

本章的核心 claim 是：Fable 5 把人与模型的关系从短轮次提示词推进到长程任务委托。机制上，模型可以利用更大的上下文、跨过临时阻塞、搭建支架并继续推进。实践上，委托者必须把验证、记录和人工复核写进任务定义，否则长程能力会变成长程风险。

3 第二章：Sonnet、Opus 与 Fable

3.1 模型选择是一道资源分配题

Dan 在 ‘00:04:47–00:05:11’ 把问题问得很尖锐：如果把一个大任务丢给 Fable，让它跑几小时或过夜，它可能是他试过最令人印象深刻的模型；同一段里他也指出，它慢、贵，让人不想把它用在所有日常任务上。这个矛盾决定了本章主线：Fable 的正确位置来自任务复杂度、时间跨度、失败成本和验证成本的共同判断。

可以用一个简单的任务成本模型来表达这种判断：

$$C_{\text{task}} = C_{\text{model}} + C_{\text{human-review}} + C_{\text{rework}} + C_{\text{coordination}}$$

- C_{task} ：完成一个任务的总成本，包含金钱、时间和注意力。
- C_{model} ：模型调用本身的成本，包括价格和等待时间。
- $C_{\text{human-review}}$ ：人类审查、理解、验收和签字的成本。
- C_{rework} ：模型误解、漏做、绕行失败之后返工的成本。
- $C_{\text{coordination}}$ ：多代理、多会话、多上下文之间同步和交接的成本。

如果任务很小，Fable 的 C_{model} 可能压过其他收益；如果任务很大，Fable 降低 C_{rework} 和 $C_{\text{coordination}}$ 的价值可能更高。这里的判断像派一个资深工程师去做事：让他处理跨系统迁移、架构权衡和长时间排障很合理；让他回答一个 NBA 快问就显得资源错配。

Fable-worthy question

所谓 *Fable-worthy question*，指的是一个资源判断：任务是否需要深思、长上下文、多步执行、绕过阻塞、产出证据，并且这些收益足以覆盖慢和贵。

3.2 前置规划、文档和团队对齐

Mike 在 ‘00:05:11–00:06:32’ 讲到，他现在会和 Fable 做更多架构规划对话。Instagram 的经历让他对不同阶段需要多少基础设施复杂度有直觉；他会追问：如果这个东西很快要上线，是否应该考虑不止一台服务器，是否需要更成熟的抽象。随后，他还会让模型把刚才的讨论整理成 HTML 页面、Markdown 文档或带图的材料，用来和团队对齐（‘00:06:14–00:06:32’）。

这说明 Fable 的价值不限于“把代码写出来”。它也在需求、架构和沟通层面工作：把口头讨论压缩成可共享文档，把隐含取舍显式化，把团队早期对齐提前。对于工程团队而言，这类文档属于工程工作的一部分；当构建速度上升时，早期对齐会变成减少返工的关键杠杆。

为什么规划材料变重要

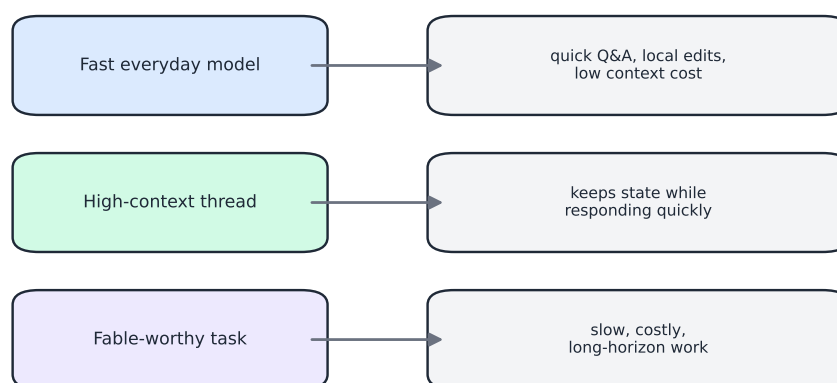
构建速度越快，错误方向的损失也越快扩大。一个 HTML 原型或带图 Markdown 文档，相当于把抽象想法变成可讨论的对象。团队可以围绕它指出误解、补充约束、调整架构，而不用等到完整实现之后再发现方向偏了。

3.3 subagents 与主线程响应性

Mike 在 ‘00:06:58–00:07:29’ 进一步描述执行方式：先做规划和对齐，然后让模型在白天或夜里执行任务块；这意味着他会开更多并发会话。具体做法有两种：一种是一个长时间运行的 Claude Code 会话，通过 background 的 forked *sub agents* 去做任务，让主线程保持 responsive；另一种是在需要集中推进时开五六个标签页，分别处理长而完整的工作。

这段话解释了为什么“代理编排”会成为新技能。并发会话会带来吞吐量，也会带来上下文分叉、冲突合并、责任归属和验收顺序问题。好的主线程像项目经理：它保持高上下文、快速响应、决定何时启动后台任务、何时等待下一轮反馈。后台代理像分工执行者：它们完成清晰边界内的工作，并把证据和限制带回来。

Model selection by task shape



Good selection asks: how much context, autonomy, latency, and review does this task need?

图 2: 按任务形状选择模型与执行面: 快问、低风险查询和局部编辑适合更快模型; 跨系统、长上下文、长程执行和证据生成更接近 Fable-worthy task。此图根据 ‘00:06:58–00:10:02’ 中关于主线程响应性、后台 subagents、effort levels 和 Sonnet/Fable 选择的讨论重绘。

3.4 effort levels、Sonnet 与 Opus 的谨慎比较

Mike 在 ‘00:08:03–00:08:34’ 承认, 对于很细的交互问题或 UI 微调, Fable 会 “想得很深”。他因此第一次更认真地调 *effort levels*, 例如把一个 UI tweak 放到 medium 看结果; 他还说自己在 Opus 上较少这么做, 因为他感觉 Fable 的 effort range 更宽。这里的比较要谨慎: 自动字幕把早先 Opus 版本识别为 “Opus 4 5 or Opus 46”, 仅凭本地字幕无法确认具体版本号。因此正文只写 Opus 系列, 避免扩展成确定版本。

在 ‘00:08:34–00:10:02’, Dan 用 “火箭筒打蚊子” 的类比追问: 移动中遇到快问, 是否还会用 Fable。Mike 的回答很实在: 他曾经这么做, 后来遇到 NBA Finals 之类快问时, 把 iOS app 切到字幕疑似识别为 ‘son’ 的模型。结合 outline 的术语表, ‘son’ 在模型选择语境下谨慎按 Sonnet 处理。Mike 还指出, 差异的核心在答案需要多少 thinking; tokens per second 只解释一部分体验, 有些答案本来就无需完整深思。

ASR 与模型名不确定

本段字幕是自动英文字幕, 存在明显识别误差: ‘cloud’ 在 Claude Code 语境下应规范为 Claude; ‘son’ 很可能指 Sonnet; ‘Opus 4 5 or Opus 46’ 不能直接写成确定版本号。本文因此保留模型名限定, 只把上下文明确的说法纳入正文。

一条实用选择规则

日常快问、低风险查询、轻量 UI 微调, 优先考虑更快、更便宜的模型或较低 effort; 跨系统实现、长程任务、架构规划、需要绕过阻塞并生成证据的任务, 才更接近 Fable 的优势区。

Mike 在 ‘00:09:27–00:09:55’ 还提出一个产品层面的方向: 用户不该被迫频繁思考模型选择; 长期看, 可能需要按使用场景分桶, 或者让不同 surface 保持 *sticky model selection*。这很关键, 因为

模型梯队如果只靠用户每次手动判断，会把认知负担转嫁给用户。更好的产品形态会把“快问、深思、长程执行、后台代理”这些模式做成可理解的入口。

3.5 本章小结

本章的核心 claim 是：Sonnet、Opus 与 Fable 的比较应服务于任务选择，避免滑向模型声望排序。机制上，模型选择取决于总任务成本 C_{task} 、所需 thinking、上下文长度、执行跨度和验证负担。实践上，团队需要建立模型使用梯队：快问题走轻量路径，长程委托走高能力路径；对 ASR 中不确定的模型名，只保留可由上下文支持的层级关系，具体版本号保持来源限定。

4 第三章：周末媒体追踪器：agent-native 架构的最小样本

这一章的核心结论很直接：Mike 展示的媒体追踪器提供了一个缩小版样本，说明软件可以怎样把 Claude 放进自己的使用、编辑、预览和继续演化流程里。视频在 ‘00:10:06–00:15:00’ 的讨论，把抽象的 agent-native 落到一个普通需求上：一个人想追踪游戏、剧集、电影推荐，Claude 负责构建、搜索、录入、修改和展示预览，人负责提出意图、看结果、继续使用。

4.1 周末上下文：为什么这个例子有教学价值

主持人在 ‘00:10:02–00:10:05’ 追问 Mike 能否展示自己用 Fable 5 做出的东西。Mike 的回答先把场景放在周末：他们鼓励团队使用个人账号，尤其在周末尝试个人项目；他也强调这次刻意离开 Anthropic 内部工具，回到更朴素的 pure Claude Code 工作状态，在周末做一个东西（‘00:10:07–00:10:28’）。

这个上下文很重要。周末项目通常具有三种约束：时间碎片化、需求高度个人化、可用性比形式完美更关键。Mike 后面补充说，那个周六早上他还有一整个周末的孩子活动，工作方式变成 “kick off work”，带孩子去徒步，回来继续推进，甚至偶尔在路上用 remote mode 看进展（‘00:11:49–00:12:08’）。这正好检验长程代理的一个现实能力：人离开键盘后，任务仍然可以在明确上下文里继续推进。

为什么“周末项目”是一个好压力测试

周末个人项目像一个小型厨房实验：食材是真需求，厨师时间很碎，成品只服务少数人，反馈马上发生。这样的场景无法靠会议、排期和完整团队流程掩盖工具缺陷；如果代理真的有用，它必须能接住粗粒度意图、持续做事、让人快速看见结果，并在下一次使用时继续接力。这个测试比精心包装的 demo 更接近普通人的真实工作节奏。

4.2 入口降低：终端、桌面 app 与非工程师用户

Dan 在 ‘00:10:28–00:10:31’ 先问 Mike 使用的是 terminal app 还是 desktop app。Mike 说自己多数时间仍在终端里，也提到 Ghostty 与 terminal app；紧接着他举了一个更有启发性的旁证：他的妻子是偏 UX designer / PM 背景的非职业工程师，却通过桌面 app 喜欢上 Claude Code，因为桌面 app 简化了一些抽象层（‘00:10:31–00:10:48’）。

这里的教学点是入口形态会改变谁能参与软件构建。终端更像后厨，适合熟悉命令、文件、进程和错误输出的人；桌面 app 更像前台工作台，把复杂操作包装成更可触达的控件与对话。对于 UX、PM、运营或领域专家，门槛下降意味着他们可以把“产品判断”更直接地传给构建系统，而无需先把自己训练成全职工程师。

代理原生架构的第一层：让领域意图进入系统

一个 **agent-native** 产品首先要提供意图入口：用户可以用自然语言、链接、上下文片段或界面操作表达目标。这个入口越贴近日常工作语言，非工程师越能参与构建和调整。Mike 的例子说明，桌面 app 的意义在于降低抽象成本，让 UX/PM 这类角色也能进入 Claude Code 的循环（‘00:10:36–00:10:46’）。

4.3 个人媒体追踪需求：从推荐碎片到可维护清单

Mike 展示的产品需求很朴素：他玩游戏、看电视剧，也不断收到朋友或平台推荐，于是想要一个 **media tracker**，把这些游戏、剧集、电影和后续推荐收进一个适合自己的系统（‘00:10:57–00:11:16’）。这类需求很常见：信息分散在聊天、网页、视频平台、口头推荐和脑中印象里，人真正需要的是一个能快速补全条目、查找相关信息、提醒后续内容的私人索引。

他给这个工具定的第一个标准是“really easy to add things”。具体做法是用户可以直接和 Claude 说话，Claude 进行 **agentic search**，把正确条目放进去；如果有新一季或某个游戏续作，Claude 还可以主动去研究相关信息（‘00:11:17–00:11:33’）。这里的产品核心是“把输入摩擦降到最低”：用户只要表达“我想记住这个东西”，代理负责补全字段、来源和分类结构。

未知风险：自动补全也会引入错误

媒体条目看似低风险，依然会遇到同名作品、地区译名、续作编号、发布日期和平台可用性差异。Claude 的 **agentic search** 可以减少人工查找成本；如果工具没有来源记录、人工确认和后续修正入口，错误会悄悄进入个人知识库。画面证据显示了 Library、Ask Claude 和 Dev 记录，来源候选与人工确认机制仍需要从产品设计层面补足。

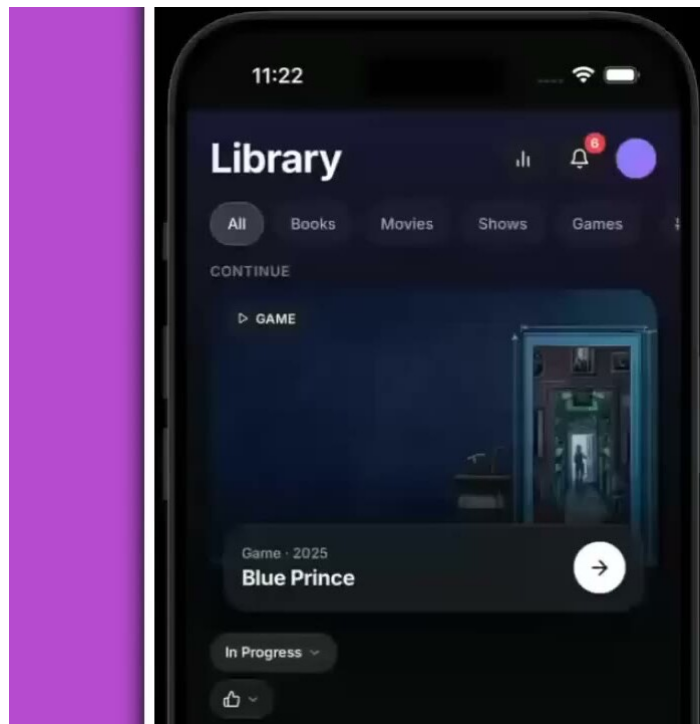


图 3: 媒体追踪器的 Library 首页：游戏、电影、剧集等内容被组织成可继续推进的个人清单。¹

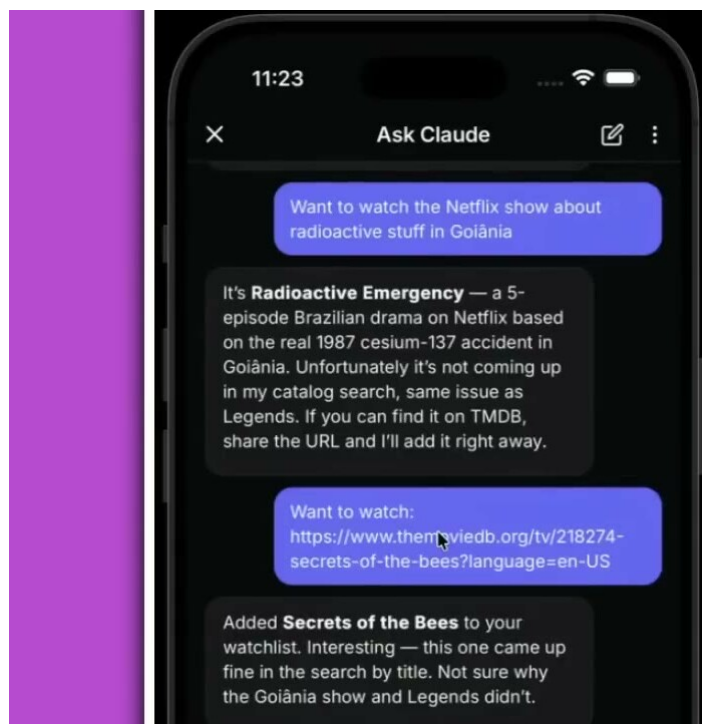


图 4: Ask Claude 界面：用户用自然语言和 URL 给出意图，Claude 负责检索、识别并把条目加入 watchlist。²

4.4 把软件团队靠近软件：agent-native 的最小结构

Mike 随后把这个小工具上升到 Labs 今年一直思考的问题：怎样把“software team”，也就是当下越来越多由 Claude 承担的构建能力，带到软件本身附近（‘00:11:39–00:11:49’）。这句话是本章的关键。传统 app 把用户、应用、开发团队分成三层：用户提需求，开发者改代码，应用下一版才变化。Mike 的原型尝试把这三层拉近：用户在 app 里表达需求，代理在 app 能触达的工具和上下文里执行，用户通过预览继续判断。

他把这个尝试称作一个 spike：能否让软件“from within itself”发生修改（‘00:12:10–00:12:20’）。他做了 React Native 版本，也做了 web 版本；已有的聊天入口可以让用户通过 URL 添加内容（‘00:12:20–00:12:33’）。他还说，希望自己永远无需为了操作而翻菜单，这个愿望把 **agent-native architectures** 推到一个极简形式：让代理可以修改 app，并让产品里的每个关键东西都能被代理访问、调用和操作（‘00:12:33–00:12:58’）。

本章给出的工作定义

本章把“代理原生架构”定义为四件事的组合：第一，产品有自然语言或上下文入口；第二，产品能力被暴露成可调用工具；第三，代理能在受控环境里执行编辑或数据操作；第四，人可以通过预览、差异视图或对话记录检查结果。用直观比喻说，它像一个工作室：用户说目标，助手拿得到工具和材料，临时样品能马上摆上桌，人再决定继续、修改或收下。

¹视频画面时间区间：00:10:57–00:11:30。

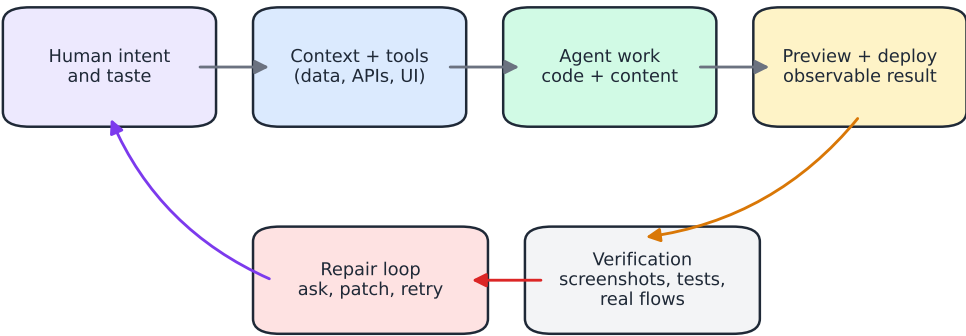
²视频画面时间区间：00:12:27–00:13:13。

可以把这个循环压缩成一个轻量模型：

Intent → Agent Action → Preview → Human Judgment → Next Intent

其中, Intent 是人的目标表达, Agent Action 是 Claude 通过工具执行搜索、录入或代码编辑, Preview 是可见结果, Human Judgment 是人的产品判断, Next Intent 是下一轮改动。这个模型来自 ‘00:12:10–00:14:18’ 的连续示例，适合解释小型个人工具，也能推广到后续章节的团队工作流。

Agent-native product loop



The model matters, yet the loop is what makes the work usable.

图 5: 代理原生产品循环：人的意图和品味进入上下文与工具层，代理完成代码和内容工作，再通过预览、验证和修复回路回到人。此图根据 ‘00:11:17–00:14:50’ 中关于媒体追踪器、Claude 工具调用、编辑请求、预览和自修改产品行为的讲解重绘。

4.5 编辑请求与实时预览：产品自己长出修改回路

在 ‘00:13:13–00:13:20’，Mike 明确提出下一步兴趣：移动中从软件内部修改软件。具体机制是长按一个小聊天入口；他先说 “what I built”，又马上修正成 “what Claude built”，说明 Claude 在构建中承担了真实开发者角色（‘00:13:20–00:13:24’）。

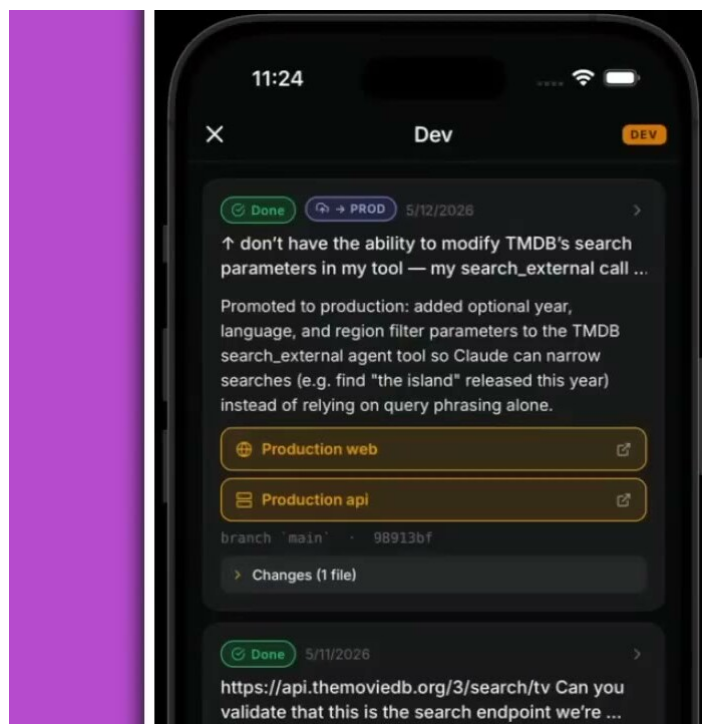


图 6: Dev 视图中的完成记录和生产链接：代理接收修改请求、生成变更、暴露预览与生产入口，形成可观察的闭环。³

这个入口背后的实现是使用 managed agents 接受 `edit requests`，然后让用户预览结果；Mike 还提到用了 Vercel 的 `live preview`（‘00:13:24–00:13:36’）。这个功能本身也来自一次 one-shot 构建，后来他持续添加能力。它甚至提供 `diff view`，用户也可以进入 managed agent conversation 查看代理做了什么（‘00:13:36–00:13:48’）。

live preview 与正式验证的区别

`live preview` 解决的是“我能否马上看到改动效果”。正式验证还需要更多证据：边界条件、回归路径、设备差异、权限控制和失败恢复。本章的媒体追踪器是个人工具，所以 Mike 对长期代码质量和维护性要求较低（‘00:13:48–00:13:58’）。一旦同样模式进入团队生产系统，预览循环必须补上测试、审查、日志和回滚机制。

最具体的编辑例子出现在 ‘00:14:00–00:14:18’。Mike 说，他在移动中提出过一个 feature request：原生 iOS 上浮动操作按钮（FAB）位置太低，其他环境显示正常；借助 Expo tooling，改动可以在手机上 `live reload`。这个细节很小，教学价值很大：agent-native 的收益经常体现在“现场发现、现场表达、现场预览”的微循环里。传统流程里，这类小问题容易被记在脑中、拖到电脑前、写成 issue、再等待修复；这个原型把问题、上下文和修改入口放在同一个使用场景里。

³视频画面时间区间：00:13:17–00:13:58。

“自修改”需要精确定义

本段材料支持的说法是：这个原型把编辑请求、managed agents、预览、差异视图和代理对话记录接进了产品体验（‘00:13:13-00:13:58’）。更强的“软件完全自主修改自身并上线”需要权限、审查、回滚和责任边界，本段字幕没有提供完整证据。Dan 在 ‘00:14:54-00:15:00’ 还专门把 self-modifying part 作为一个需要单独对待的边界。

4.6 为什么这是最小样本：生产化能力还要补课

Mike 在 ‘00:14:21-00:14:37’ 很清楚地限定了这个工具的规模：它是服务个人需要的原型，生产级系统面向百万用户时还要补上治理、质量和责任机制。它让人兴奋的地方在于，项目没有停在周末结束那一刻；Mike 可以继续在使用中推进它，并拥有一种 end-to-end close 的感觉。换句话说，个人工具从“一次性做出来”变成“在日常使用里继续生长”。

这也解释了它为什么能作为 agent-native 架构的最小样本。最小样本追求闭环完整：个人需求进入系统，Claude 负责构建和操作，界面提供预览，用户继续提出改动；完整企业治理属于下一层能力。Mike 把它视为 Fable 构建能力的体现，也把它和“Claude 如何嵌入软件，超越单纯使用侧”联系起来（‘00:14:37-00:14:50’）。

对读者而言，这里有一个容易漏掉的隐含前提：代理进入产品后，产品经理、设计师和工程师的边界会变薄。产品会从等待下个版本发布的静态对象，逐渐接近一个可对话、可委托、可预览、可审查的工作环境。这个方向很有力量，同时也会把权限管理、工具暴露、数据来源、测试覆盖、UI 可解释性和人工责任推到更核心的位置。

从这个例子抽出的实践原则

如果要设计一个小型 agent-native 产品，可以先问五个问题：用户如何表达意图；代理能访问哪些数据与工具；代理的动作怎样被限制；用户如何预览和比较结果；产品如何记录代理做过什么。媒体追踪器的价值在于，它用一个个人需求同时展示了这五个问题的雏形（‘00:11:17-00:14:18’）。

4.7 本章小结

本章的核心 claim 是：周末媒体追踪器展示了 agent-native 架构的最小可理解形态，重点在于把用户意图、Claude 构建能力、产品内部工具和预览反馈连接成一个闭环。它的机制是自然语言入口加工具调用，再加 edit requests、live preview、diff view 与 managed agent conversation 这些可观察环节。它对构建者和团队的启示很硬：当软件允许代理在使用现场继续工作，产品设计就必须同时考虑入口、权限、验证、责任和后续维护。

5 第四章：成本坍塌：从 Instagram v1 到个人工具

上一章的媒体追踪器给出了一个最小样本：个人可以把意图、工具访问、修改请求和预览闭环交给 Claude 推进。第四章解释这个样本背后的经济结构：Fable 5 让“想法到可用产物”的任务级成本明显下降。Dan 在章首把问题放到 Instagram 早期来比较：同样是做一个真实可用的小产品，过去需要投入多少人、多少夜晚、多少协调成本；现在，个人可以把更多执行环节交给模型推进（00:15:02-00:15:16）。

5.1 从 Instagram v1 看见时间尺度的变化

Mike 回忆早期 Instagram 时, 先强调自己当时已经是高产程序员, 熟悉移动开发, 并且团队对要做什么很清楚 (00:15:16–00:15:32)。即使具备这些条件, 从一个想法到一个 fully realized version, 仍然大约需要四天左右的通宵工作 (00:15:32–00:15:41)。他随后把 Instagram v1 作为参照: Instagram v1 的功能更多, 整体规模仍然接近前一章提到的个人媒体追踪器; 当时需要他做前端和后端, Kevin 做初始滤镜, 连续几天熬夜才能推出 (00:15:49–00:16:04)。

更准确的读法是: 过去只有非常强的工程能力、清晰的产品判断、配合默契的团队、已有平台经验同时存在时, 一个小产品才有机会迅速落地。今天, Fable 5 这类模型把其中一部分执行负担吸掉, 使个人工具、小团队原型、内部业务系统的起步门槛下降。

成本坍塌的精确定义

本章所说的 cost collapse (构建成本坍塌), 指任务级总成本下降: 从想法到产物的时间变短, 沟通损耗降低, 返工次数减少, 迭代摩擦变小。它并未宣称模型调用免费, 也没有取消人类对质量、风险和取舍的责任。

Instagram 例子还有一个容易被忽略的背景: Mike 说那套能力建立在多年 iOS 经验之上 (00:16:07–00:16:12)。这说明旧时代的“快”, 本身已经是长期技能沉淀后的快。Fable 5 改变的是可委托的执行面: 过去需要人亲自把前后端、数据、界面、部署、调试串起来; 现在, 模型可以承担更多串联动作, 人类把注意力转向目标、边界和验收。

5.2 想法到产物: 真正下降的是任务级成本

Mike 说 Instagram 发布之后, 团队手里有很多下一步想法, 现实瓶颈却常常变成“先保持站点可用”或“先加一个增量功能”; 一个 hashtag 功能都可能要一周 (00:16:14–00:16:31)。这段话解释了成本坍塌的第一层机制: 真实构建成本来自整条任务链, 包含上线后的修补、功能优先级、协作排期、验证和再次沟通。

为了把这个经济学直觉说清楚, 可以用一个简单的任务成本模型:

$$C_{\text{task}} = C_{\text{model}} + C_{\text{human-review}} + C_{\text{rework}} + C_{\text{coordination}}$$

各个符号的含义如下:

- C_{task} : 完成一个任务的总成本, 包含模型、人员、返工和协作的总投入。
- C_{model} : 模型调用、工具运行、算力和相关服务的直接成本。
- $C_{\text{human-review}}$: 人类审查、验收、读代码、试用产品、判断风险所花的成本。
- C_{rework} : 由于理解偏差、实现错误、边界遗漏、质量欠佳导致的返工成本。
- $C_{\text{coordination}}$: 找人、排期、交接背景、解释需求、等待反馈和组织资源的成本。

这个式子解释了为什么单次模型价格会误导判断。一个更贵的模型如果显著减少返工、压缩等待、提升一次完成率, 任务级 C_{task} 可能下降。相反, 一个便宜模型如果造成多轮修补、反复解释和更高审查压力, 总成本会升高。

别把“模型贵”误读成“任务贵”

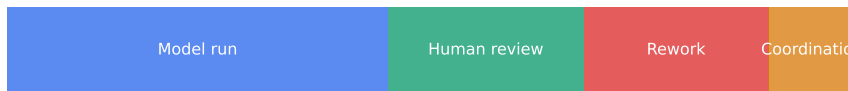
成本评估的单位必须是任务。只盯 C_{model} 容易忽略 $C_{\text{human-review}}$ 、 C_{rework} 和 $C_{\text{coordination}}$ 。Fable 5 的价值只有在它降低任务总成本时才成立；如果任务简单、风险低、上下文短，较轻模型或传统工具仍可能更划算。

Task-level cost, not per-turn price

Older build path



Fable-worthy task path



$$C_{\text{task}} = C_{\text{model}} + C_{\text{review}} + C_{\text{rework}} + C_{\text{coordination}}$$

图 7: 任务级成本栈：更强模型的单次运行可能更贵，但如果减少人工复查、返工和协调损耗， C_{task} 仍可能下降。此图根据 ‘00:15:00–00:18:53’ 中关于 Instagram v1、个人工具和构建成本坍塌的讨论重绘。

5.3 协调损耗：成本公式里的关键分量

在上面的公式里，最容易被低估的是 $C_{\text{coordination}}$ 。Mike 把“想法到执行”的缝隙说得很具体：一个有想法的人，过去可能要去找咨询公司承接，实现过程会把原本脑中的意图压缩、转述、排期、再解释；这个过程很 lossy，意思是信息在交接中不断丢失（00:16:51–00:17:12）。这里的损耗像拍一张清晰照片后反复截图转发：每一层都还能看见大概内容，细节、色彩和边缘逐步变差。

Fable 5 的关键变化在于 closing that gap between intent and execution（缩短意图与执行之间的距离）。Mike 明确说，让模型更自主只是其中一部分，更让人兴奋的是它让非传统 builder 也能构建真实东西（00:17:18–00:17:34）。这条线索非常重要：技术工具的扩散价值来自“让更多人能完成以前只能交给工程团队排期的任务”。

什么是 intent-to-execution gap

intent-to-execution gap 指“人脑里的目标”和“世界里的可运行产物”之间的距离。这个距离由需求表达、背景传递、工具权限、实现技能、调试能力和验收标准共同决定。模型越能读取上下文、调用工具、保持长程执行，距离越短。

产品判断仍要由人负责。想法本身仍要被澄清：谁使用、解决什么痛点、失败时有什么风险、怎样验收、何时停止。模型降低的是执行阻力，人类仍负责定义值得执行的方向。

5.4 内部招聘工具：协调成本下降的证据

最有说明力的例子来自 Anthropic 内部。Mike 提到，有人给他发消息，因为他们做了一个内部工具，把 Fable 和内部 MCP 访问结合起来 (00:17:52–00:18:08)。这位同事在招聘相关工作中使用它，并说这是自己人生第一次感觉“脑中的东西”和“世界中存在的东西”紧挨在一起，可以直接做出来 (00:18:08–00:18:19)。

MCP 在这里可以理解为“模型拿到合规工具接口的方式”。如果把模型比作一个会思考的同事，MCP 就像给这位同事一串受控钥匙：可以访问招聘流程、内部数据、消息系统或业务工具，同时还需要权限边界和审计。没有这类工具接口，模型只能聊天；有了工具接口，模型才能把意图推进成工作流。

高信号原话 (00:18:08–00:18:19)

Mike: It is the first time in my life where the thing that's in my head and the thing that exists in the world are right next to each other. I can just do it.

这段话的教学价值在于，它把成本坍缩从“工程师更快写代码”扩展到“领域工作者更快制造工具”。过去，招聘同事想要内部工具，常见路径是凑合使用现有系统，或者请求内部工具工程师；后者往往背着几十个需求 (00:18:23–00:18:36)。现在，她可以在 Fable 和 MCP 的支撑下亲自推进。成本下降的来源是执行链缩短，也是组织等待减少。

个人工具的战略含义

当一个业务人员能把自己的上下文、数据入口、工作判断交给模型，并持续验收结果，软件生产会从“少数工程团队服务多数需求”扩展成“更多领域人员直接制造自己的工具”。这会提高产出，也会把权限、数据边界和质量审查推到更前面。

5.5 成本坍缩的风险：可建造性提高，判断压力也提高

Mike 在这一段给出乐观判断：人的创造能力和可实现空间很大；模型正在让更多人把想法推进到“感觉真实”的东西 (00:18:39–00:18:53)。这句话需要配上风险说明。构建变便宜后，低质量工具、权限误用、流程碎片化、无人维护的小系统也会变多。组织需要回答三个问题：

- 谁拥有这个工具的目标和验收标准？
- 这个工具接触了哪些内部系统、客户数据或招聘数据？
- 当模型生成的流程出错时，谁修、谁停、谁向受影响的人解释？

这正是下一章“软件工程是否结束”的入口。构建成本下降没有消除工程问题，只是把工程问题从“谁来写第一版代码”推向“谁来定义边界、验证质量、承担后果”。

5.6 本章小结

本章的核心 claim 是：Fable 5 带来的 cost collapse 发生在任务级成本层面，单次调用价格只能解释局部体验。机制是模型压缩 idea-to-artifact 的时间，降低协调损耗，并让领域人员通过 Fable 和 MCP 更接近自己的工作意图。这个判断成立依赖四个前提：模型能保持足够长的上下文，工具访问受到权限治理，人类有能力复查结果，组织愿意为维护 and 责任闭环付成本。实际含义是，个人工具和内部业务系统会大量增加，团队必须同步建立权限、验收、维护和责任规则。

6 第五章：软件工程结束了吗？职责边界正在重画

成本坍塌自然带来第五章的问题：如果模型能显著压低实现成本，软件工程师的职责还剩什么。Dan 直接提出听众脑中的问题：given everything you just said, is software engineering over? (00:18:53–00:19:05)。Mike 的回答并没有给出一句口号。他的意思更细：软件工程的手感、工具、分工已经剧烈改变；软件生产和软件开发仍然非常活跃，而且依旧关键 (00:19:07–00:20:12)。

6.1 旧定义：工程师把复杂性压进编辑器

Mike 说，如果在 Instagram 时代问他“什么是软件工程”，他大概会说：思考困难问题，设计架构，然后在 TextMate、文本编辑器或 Xcode 里花大量时间修改代码 (00:19:07–00:19:30)。Dan 顺手提到 watching Rails, Mike 接着补充 Django 层的细节理解、部署后的 bug 修复 (00:19:30–00:19:41)。这些词很有时代感：TextMate、Xcode、Rails、Django 都代表一种工程师身份认同，即通过亲手理解框架、亲手修复细节、亲手把系统推出去来获得掌控感。

这套 craft (手艺) 仍然值得尊重。它训练了抽象能力、调试耐心、系统感、审美和责任意识。变化在于，Fable 5 这类模型开始接管其中一部分“亲手打字、亲手查接口、亲手追 bug”的工作量，让工程师的价值中心向上移动。

craft 的两层含义

craft 可以指“手上功夫”：熟悉编辑器、框架、调试器、语言特性；也可以指“判断功夫”：知道什么值得做、什么质量算好、什么风险能接受、什么技术债会伤人。模型首先冲击的是手上功夫，判断功夫仍然需要人类长期训练。

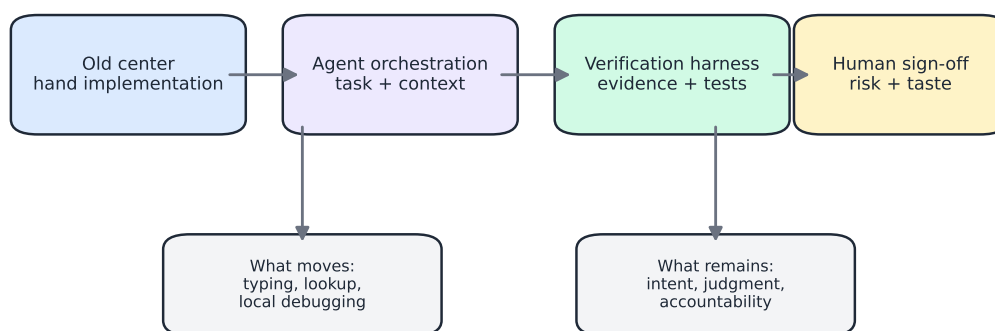
6.2 职责链条：信任半径扩大，角色边界重画

Mike 观察到，许多旧式工程活动正在 collapsing into other parts of product management，产品经理和工程师之间的边界也更 diffuse (00:19:41–00:19:56)。这句话需要放进一条职责链里看：模型的信任半径扩大，PM 和工程师的边界随之重画，错误也会沿着更长的代理链条传播。过去，PM 负责需求、优先级和用户理解；工程师负责架构、实现和上线。代理模型进入之后，PM 可以直接让模型做原型，工程师也要更深地参与产品意图、验证路径和用户体验。

职责边界正在重画，至少包括四个方向：

- 意图设计：把模糊愿望改写成可执行目标、上下文、约束和验收标准。
- 执行编排：决定哪些任务交给 Fable，哪些任务交给较轻模型，哪些任务由人亲自处理。
- 验证支架：设计测试、截图、日志、真实流程和回归路径，让代理产物能被检查。
- 责任闭环：上线前读懂变化，上线后处理事故，向用户和团队解释取舍。

Software engineering role shift



Capability shifts the work upward; responsibility still lands on a person.

图 8: 软件工程职责重画: 模型把打字、查接口、局部调试等执行动作上移到代理编排层, 人类价值集中到意图、验证、品味、风险判断和问责。此图根据 ‘00:19:03–00:21:48’ 中关于软件工程手艺、Fable 信任半径和 human endeavor 的讨论重绘。

软件工程的新核心

软件工程的重心正在从“人类完成大部分实现动作”移动到“人类定义目标、分配代理、设计验证、判断质量并承担结果”。代码仍然重要, 系统仍然要运行, 最终签字的人仍然需要理解自己交付了什么。

6.3 风险链条: 错误会走得更远

Mike 说, Fable 是一个重要台阶, 因为他愿意把更高信任放在模型身上: 模型可以 see things through (把事情推进到底), 甚至能 reasonably architect things (做出相对合理的架构) (00:20:17–00:20:35)。这里的“信任”要谨慎理解。信任半径扩大, 意思是模型可承担更长的链条、更复杂的上下文、更少中途打断的执行任务。它仍然需要边界、日志、审查和验收。

可以把它想象成从“会回答问题的实习生”变成“能做一段项目的初级同事”。前者适合问答和局部片段, 后者可以拿一份任务说明、查资料、改代码、跑检查、返回证据。即使如此, 负责人仍要看结果是否符合产品意图、是否引入隐性风险、是否值得上线。

信任半径扩大带来的隐藏风险

代理能独立走得更远, 错误也可能扩散得更远。长程执行需要更强的验收边界: 任务说明要写清楚, 权限要最小化, 关键路径要可复现, 输出要附带证据。否则, 模型越能干, 错误越难在早期被发现。

6.4 人类仍负责 taste、judgment 和 accountability

Mike 把剩下的核心说成 very human endeavor: 你要决定自己到底要产出什么, 产物是否真的好 (00:20:41–00:20:53)。这就是 taste (品味)、judgment (判断) 和 accountability (责任) 的集合。

品味决定“这个界面、流程、抽象、功能是否优雅”；判断决定“这个方向是否值得做、这个风险是否可接受”；责任意味着结果出了问题，人类负责人要能解释、修复和承担。

这也是成本坍缩之后最大的 unknown unknown。很多团队会以为自己获得了更多工程产能，随后才发现瓶颈迁移到了判断质量：需求说得含糊，模型也能交付一个看似完整的产物；权限给得太宽，内部工具就可能接触过多数据；验收只看 happy path，边界场景会在真实用户那里爆炸。

判断力成为稀缺资源

当模型降低实现门槛，最稀缺的东西会变成清晰意图、好品味、风险判断和愿意负责的人。工程师的训练重点也会改变：少一点机械打字崇拜，多一点系统边界、验证设计和产品责任。

6.5 情感维度：失落与扩展产出同时存在

Mike 没有回避情感损失。他说，很多人热爱“亲手把问题优雅解决”的感觉；他自己也喜欢那种梦里还在想代码、早上醒来突然想出优雅解法的体验（00:21:01–00:21:17）。随着模型承担更多实现动作，这种体验的一部分会消失。他提到，一些优秀工程师会有 feeling of loss，同时也会感到自己现在能完成 insane amounts of work（00:21:17–00:21:29）。

情感张力（00:21:17–00:21:40）

Mike: There is a feeling of loss, as well as the feeling of, oh my god, I can do insane amounts of work now.

Dan: It is normal to feel sadness for that kind of thing and excitement.

这段对话很重要，因为它避免把技术变化包装成单一情绪。对工程师来说，旧 craft 的消退是真实损失；对产品和组织来说，产出边界扩大也是真实收益。成熟的团队需要同时承认两件事：一方面，保留工程师对质量和系统的敬畏；另一方面，训练他们使用代理放大自己的产出。

6.6 本章小结

本章的核心 claim 是：软件工程没有结束，工程职责正在从亲手实现转向意图、编排、验证、质量判断和责任闭环。机制是 Fable 5 扩大模型的信任半径，让更多实现与架构动作可被委托，同时把 taste、judgment 和 accountability 推到更中心的位置。这个判断依赖的前提是：代理确实能完成较长任务，人类负责人能读懂关键取舍，组织能提供验证支架和合并前追问。实际含义是，优秀工程师需要接受手艺结构的变化，把失落感转化为新的产出能力和更严格的工程治理。

7 第六章：Anthropic 团队今天怎样协作

这一章回答一个很现实的问题：当一个团队已经把 Fable 5 这类长程代理放进日常工程流程，协作会变成什么样。Mike 的回答给出一条清晰主线：人类仍然保留产品意图、上下文、风险判断和最终责任；Claude 会话则变成并行执行、验证、反馈和原型探索的工作单元。原字幕在本章多次把 Claude 误写为 cloud；本节按语境统一写作 Claude。

本章主张：团队协作的单位从“人做任务”扩展为“人负责一片工作区，多个代理推进子任务”

Anthropic 的实践可以压缩成一句话：一个 DRI 持有目标、约束和签字责任，多个 Claude 会话承担并行推进、代码修改、PR 反馈处理和证据生成。这个结构提高吞吐量，也把新的管理成本带进团队：队列可视化、上下文回收、验证证据、生产风险和合并前问责都必须被系统化。

本章特意排除 00:25:37–00:26:53 的赞助内容。该段谈 AI observability 平台，主题接近“生产系统”，来源性质却属于广告，不能作为 Anthropic 团队实践的证据。

7.1 DRI：代理并行之后，人类责任更清楚

Mike 从组织方式说起：团队会先一起讨论产品下一步怎样演进，然后把方向拆成多个 ownership areas（责任区）⁴。这一点看似普通，实际是代理时代的关键约束。Claude 可以持有大量代码上下文，却无法自然持有团队内部的全部产品意图、路线图冲突、跨产品依赖和风险偏好。Mike 明确说，人类仍然持有一部分“context beyond Claude”：产品真实意图、当前进展、管线里即将到来的产品、未来集成方式⁵。

在这个结构里，DRI 是 Directly Responsible Individual，中文可以译为“直接负责人”。它指向一个具体的人：这个人某个产品区域、功能方向或 workflow 拥有最终解释权和签字责任。DRI 的价值在于把“谁最终理解并承担这个改动”钉住。DRI 缺失时，多代理并行会迅速变成一堆看似完成的分支、PR、实验和评论；DRI 清楚时，代理输出才会被组织成能进入产品的判断链。

用一个简单公式可以帮助理解 DRI 的负载：

$$H_{\text{owner}} = I_{\text{intent}} + C_{\text{context}} + R_{\text{risk}} + A_{\text{accountability}}$$

- H_{owner} ：人类负责人的所有权负载，也就是这项工作最终落在人身上的理解和责任。
- I_{intent} ：产品意图，包含用户问题、体验方向和团队取舍。
- C_{context} ：上下文，包含现有系统、相关产品、路线图和隐含约束。
- R_{risk} ：风险接受，包含性能、稳定性、安全、维护成本和上线时机。
- $A_{\text{accountability}}$ ：问责，包含合并前确认、事故后响应和对外承诺。

DRI 与“多 Claude 对一人”

Mike 描述的比例是“many Claudes to each human”^a。这意味着人类从“亲自完成每一步”的执行者，转成“持有问题空间并调度多个执行线程”的负责人。类比到工厂：机器变多之后，班组长的工作量没有消失；他的工作从拧螺丝转向排产、验收、异常处理和质量签字。

^a00:22:32–00:22:37。

⁴00:22:00–00:22:11。

⁵00:22:11–00:22:30。

DRI plus many agents

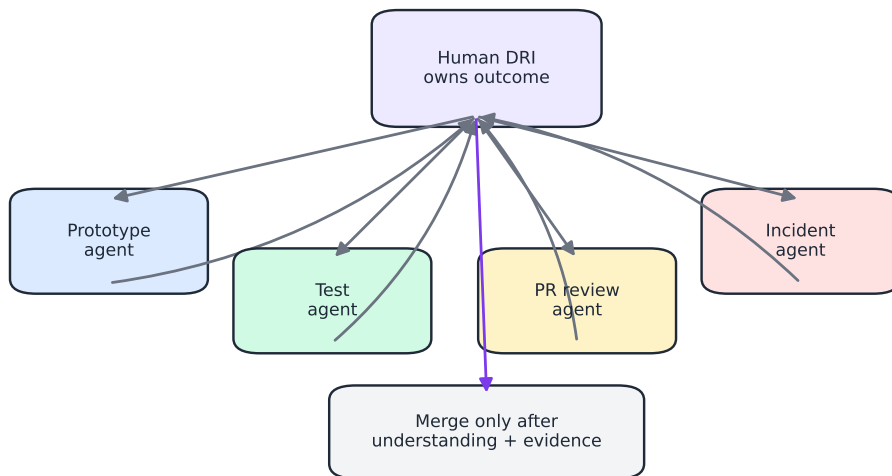


图 9: DRI 加多代理协作模型: 多个代理可以并行处理原型、测试、PR review 和事故线索, 但直接负责人仍要理解结果、检查证据并承担合并后的责任。此图根据 ‘00:22:00–00:25:35’ 与 ‘00:36:12–00:38:39’ 中关于 DRI、dashboard、PR 注意力和合并前理解的讨论重绘。

7.2 许多 Claude 会话需要新的工作面板

当每个人同时驱动多个 Claude 会话, 传统的聊天窗口会变得松散。Mike 说, 很多工程师已经各自做出某种 dashboard: 它会显示所有 Claude 在做什么、哪些任务正在等待本人、哪些 PR 需要注意, 因为人类 reviewer 或 Claude Code reviewer 已经返回了反馈⁶。它的角色接近多代理协作里的“空中交通管制塔”。

这种 dashboard 至少要解决三类问题。第一是位置感: 每个会话处在计划、实现、卡住、等待 review、需要人类决策的哪个状态。第二是注意力排序: DRI 的时间有限, 系统应把“需要本人判断”的项浮上来。第三是证据入口: PR、评论、截图、视频、测试结果和 reviewer 反馈应能从同一处进入判断。

仪表盘解决可见性, 责任仍靠 DRI 消化

dashboard 只能告诉负责人“哪里需要看”。它无法替负责人回答“这个风险能不能接受”“这个行为是否符合产品意图”“这个 PR 是否已经达到合并标准”。当代理数量上升, 真正稀缺的资源会变成人类高质量注意力。

Mike 还提到, 这类“meta maintenance of the work”会有一部分被产品标准化, 也会有一部分保持个人化, 类似不同工程师组织窗口和工作区的方式⁷。这提示了一个常被低估的设计问题: 代理工具既要提供通用队列和状态模型, 也要允许个人形成自己的节奏。强行统一所有人的工作面板, 会压扁有效的个人工作法; 完全放任个人脚本, 又会让团队协作缺少共享语义。

⁶00:23:03–00:23:27。

⁷00:23:27–00:23:43。

Many-agent work queue

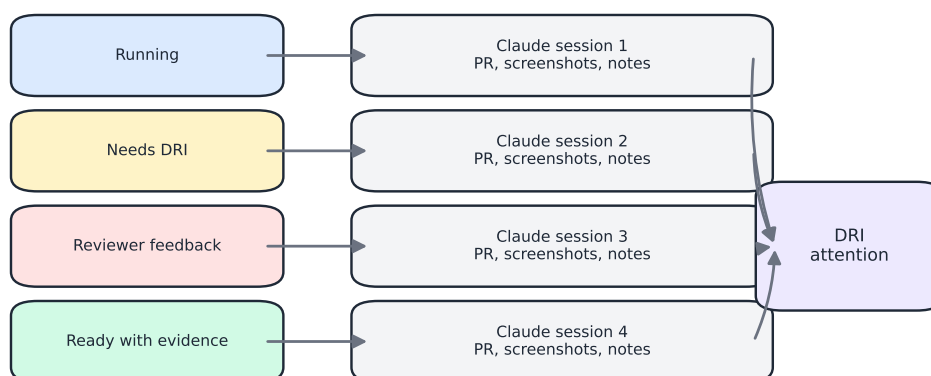


图 10: 多 Claude 会话需要新的注意力队列: 运行中、等待 DRI、收到 reviewer 反馈、已带证据完成的任务应被清楚分流。此图根据 ‘00:23:03–00:23:43’ 中关于工程师自建 dashboard、PR attention 和 reviewer feedback 的讨论重绘。

7.3 生产意识：代码写完以后，工程才刚进入风险区

Mike 随后把焦点转向 production awareness（生产意识）。他认为模型下一阶段的重要前沿之一，是理解代码部署之后发生了什么⁸。这里的“理解”包含静态仓库之外的真实运行世界：网络中断、非常规故障、扩容压力、用户路径偏移和事故响应。Mike 用 Instagram 2012–2016 年的经验作类比：很多工程工作都在处理这些上线后的复杂性⁹。

本章的关键提醒在这里：Fable 5 可以写出更多代码，也可以在代码审查中表现出系统感；生产系统仍然会把真实世界的复杂性塞回团队面前。事故响应需要人在压力下保持镇定，先收集数据，处理眼前影响，再安排长期修复¹⁰。这些动作依赖判断、优先级和组织沟通，代理可以辅助执行，责任链仍要在人类团队中闭合。

什么叫 production awareness

production awareness 指工程师理解代码在真实生产环境中的生命：部署、特性开关、数据迁移、跨服务契约、用户路径、监控、异常、回滚和事故复盘。它像建筑师理解房屋交付后的使用：图纸画完只是开始，雨天漏水、电路负载、住户动线和维修责任都会反过来检验设计。

Mike 在 00:30:23–00:31:03 又从模型能力角度补充了这个点：Fable 会在实现之后提醒“生产里这里需要不一样”，会持续追问 feature flag 是否打开，也会指出某个契约变化会影响另一处系统。这说明模型已经开始具备系统边界意识。团队应把这种提醒转化为 checklist、review prompt 和上线前证据，同时避免把它当成模型“看起来很聪明”的表演。

⁸00:23:43–00:24:02。

⁹00:24:02–00:24:15。

¹⁰00:24:17–00:24:30。

7.4 原型主导对齐：让想法先变成可讨论的东西

Mike 很喜欢 engineering prototype 在今天扮演的新角色。他强调，团队必须清楚标记某个东西是 prototype（原型）还是可上线实现¹¹。这个区分极其重要：原型的价值在于让抽象争论具象化，让团队看见交互、延迟、边界状态和用户感受；生产实现的价值在于稳定、可维护和可追责。

这里出现了一句老工程文化口号：*code wins arguments*。Mike 对这句话保持距离，因为它容易让“会写代码的人”天然赢得产品争论¹²。Fable 5 改变了争论结构：PM 也可以快速做出一个有缺陷却能表达方向的原型，然后带着“这里有八处 jank，但它展示了这个方向怎样工作”的证据进入讨论¹³。

短对话：原型怎样改变争论

Mike: The old phrase was like “code wins arguments.”^a

Mike: A PM can now say, “I just tried it ... jank in these eight ways; look, it actually shows how this could work.”^b

^a00:24:49–00:24:51。

^b00:25:10–00:25:18。

这段话的教学价值在于：代理降低了“把观点变成可体验样品”的门槛，产品讨论因此从口头偏好转向可观察证据。与此同时，原型权力也带来新风险。一个会动的原型很容易制造虚假的确定感；团队需要明确它验证了什么、还没验证什么、哪些部分只是脚手架、哪些部分已经接近生产设计。

原型对齐的正确用法

原型应服务于对齐；判断责任仍然留在团队手里。它回答“这个方向能不能被体验、能不能暴露问题、能不能打开更好的讨论”。它还没有自动回答“这个方案能否安全上线、成本是否可控、长期维护是否合算”。

Mike 最后把这一段落回 ownership：过去六个月，变化最大的是并行度和更高阶的工作抽象；保持稳定的核心仍然是 ownership¹⁴。

7.5 成本治理：有效使用比用量排行榜更重要

赞助段之后，讨论短暂转向价格和组织使用。Dan 提到 Fable 很贵，个人使用时会担心一次任务可能花掉 100 美元量级¹⁵。Mike 的回答对团队管理有启发：在专业软件团队里，模型价格应放在“结果”和“过程”里看。早期公司会挣扎于让员工采用 AI coding；随后可能用排行榜比较谁用得最多；更成熟的阶段应关注谁用得有效，并允许有效使用者获得足够预算，同时避免浪费¹⁶。

这段内容与前几章的成本坍缩相连。单次调用价格只是表面数值，任务成本还包括人类 review、返工、等待、沟通和错失机会。Mike 说，Fable 的亮点在于它经常能把任务做到满意，减少后续 9 到 10 轮“方向偏了、还要再补一块”的返工¹⁷。团队层面的结论很直接：衡量 AI 工具价值时，应

¹¹00:24:40–00:24:49。

¹²00:24:49–00:25:03。

¹³00:25:03–00:25:20。

¹⁴00:25:20–00:25:35。

¹⁵00:26:53–00:27:16。

¹⁶00:27:18–00:28:31。

¹⁷00:29:20–00:30:03。

该看端到端任务闭环，token 账单和使用次数只提供局部信号。

Mike 还补了一层个人使用的细节：周末媒体追踪器没有花到数千美元，更多是比日常多出一些用量¹⁸。真正难处理的是夹在企业预算和普通消费者之间的 hobbyist / independent 使用者：他们能从长程模型里获得真实价值，却缺少团队预算、采购流程和结果证明机制¹⁹。

7.6 代码审查：代理开始拥有“判断”，人类仍要校准

Mike 还提到一个他喜欢观察的场景：Fable 对人类或其他 Claude reviewer 的代码审查反馈作出回应²⁰。一个低质量代理会机械接受反馈，收到评论就修改。更强的代理会根据目标保真度、风险等级和系统上下文决定：这个风险可以接受；这个 reviewer 说得对；这个反馈需要反驳²¹。

这类能力非常关键，因为代码审查的核心远超“把所有评论都处理掉”。优秀工程师会判断评论背后的真实风险：有些评论指出上线阻塞，有些评论只是风格偏好，有些评论需要更多证据。Fable 开始表现出这种判断力，意味着代理可以参与审查讨论；同时，模型之间相互 review 也会制造新的校准问题。人类 DRI 需要知道哪些争论已经充分解决，哪些争论只是两个模型之间的自洽对话。

模型 reviewer 的盲点

一个 Claude reviewer 给出反馈，另一个 Fable 模型给出反驳，这仍然需要人类看见评价标准。标准包括：目标保真度、用户影响、生产风险、维护成本、测试证据和团队约定。缺少标准时，“模型看起来很有判断力”会变成新的自动化幻觉。

Mike 说，外部产品如 Claude Code 对模型团队也很有价值，因为它们形成一个“living breathing thing”：真实用户用它做多日、困难、重复的任务，反馈会进入下一轮模型改进思考²²。这说明团队实践本身也在训练产品方向：产品使用场景越真实，模型改进目标越具体。

7.7 接口变化：移动端、远程开发盒和执行地点分离

Dan 问，chat 是否仍然适合这种模型，因为 Fable 5 的使用方式更接近 delegation，逐轮问答只能覆盖其中一部分²³。Mike 没有否定消息式接口；他提出需要演进，其中第一个问题是：工作必须发生在笔记本电脑上吗²⁴。

Mike 提到 Boris 很早就把很多 Claude Code 工作转到 mobile 上，Fable-class 模型尤其适合这种方式：会话可以持续运行，Anthropic 又使用 remote dev boxes，因此人可以在移动端继续控制、补充和查看工作进展²⁵。这带来一个重要抽象：where the work is happening 与 where I am talking about the work 可以分离²⁶。

¹⁸00:28:53–00:29:03。

¹⁹00:29:10–00:29:14。

²⁰00:31:03–00:31:14。

²¹00:31:14–00:31:41。

²²00:32:02–00:32:33。

²³00:32:33–00:32:45。

²⁴00:32:45–00:33:02。

²⁵00:33:03–00:33:34。

²⁶00:33:34–00:33:39。

控制面与执行面

可以把代理工作分成两层：执行面是代码真正运行、测试执行、文件修改发生的环境，例如 remote dev box；控制面是人类发出意图、查看状态、批准下一步的界面，例如手机或聊天窗口。传统开发里，这两层常在同一台电脑上；长程代理把它们拆开，团队就要重新设计权限、状态同步、证据展示和中断恢复。

Control plane and execution plane

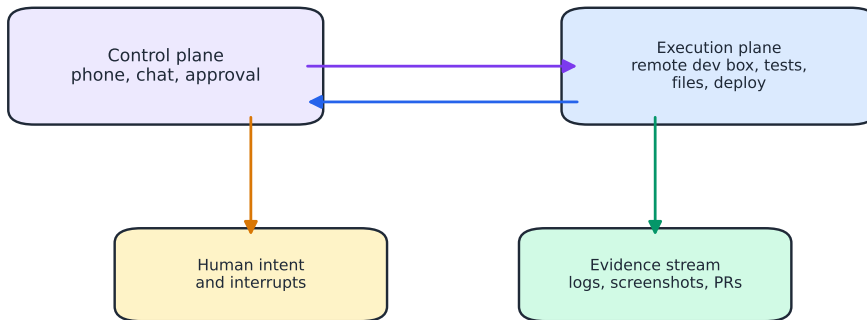


图 11: 控制面与执行面分离：人可以在手机或 chat 中给出意图、批准和中断，真实代码修改、测试和部署发生在 remote dev box 等执行环境。此图根据 ‘00:32:45–00:33:39’ 中关于 mobile、remote dev boxes 和工作地点分离的讨论重绘。

7.8 把模型上下文转回人脑

移动端和远程执行解决的是“人怎样持续控制工作”。另一个问题更难：模型做了很多决定以后，人怎样真正理解这些决定。Dan 把它说成组织内使用代理时的大瓶颈：如何把模型刚做的事情的上下文转移进人的大脑²⁷。

Mike 在接口讨论里已经提前给出线索：Fable 可能讨论、决定、提出了很多内容，团队需要把这些内容变得 comprehensible（可理解）²⁸。他们会使用一些 skills，比如让模型画图、做解释；当前 chat UI 在这里显得不足，因为它经常给出大量技术文本，读者需要投入很多精力才能完全理解²⁹。Mike 会要求 Fable 后退一步，做 more progressive disclosure of the complexity，也就是把复杂性分层揭示³⁰。

这部分对所有团队都有现实含义。代理越强，越容易一次性跨过许多中间判断；人类看见的是完成结果，却缺少路径记忆。路径记忆缺失时，后续维护、事故定位和跨团队解释都会变困难。因此，代理输出除了代码，还应产出“理解工件”：设计摘要、取舍表、状态图、PR walkthrough、风险清单、截图画廊、失败案例和 reviewer 问答记录。

²⁷00:35:41–00:36:10。

²⁸00:33:40–00:33:51。

²⁹00:33:51–00:34:12。

³⁰00:34:12–00:34:24。

上下文转移的最低标准

当代理完成一个复杂任务，DRI 至少要能回答四个问题：它改了哪些关键路径；它为什么这样取舍；它验证了哪些用户流；它留下了哪些风险。回答不出来时，工作仍处于“待理解”状态，尚未达到团队级可交付。

7.9 多人协作：从分享聊天到共享工作状态

Mike 还指出，multi-player（多人协作）仍处在早期阶段³¹。在典型情况下，一个重要工作块可能由一个人和几个 Claude 会话共同推进；这种结构还能自然流动。复杂情况会更难：事故响应里多个人同时思考；项目里多个相邻责任区同时汇合；一个独立 Claude 会话由某人启动，却需要跟上团队其他工作³²。

当前的 chat sharing 只能让人看见一部分上下文，Mike 认为还需要更强的抽象，让独立代理跟上团队整体状态³³。这属于组织协议层面的设计问题，远超界面小优化。一个代理会话如果不知道相关 PR、事故频道、产品决策、相邻责任区和最新约束，就可能在局部做得很好，在整体上制造冲突。

Mike 的判断很直白：模型已经具备 teammate collaborator 的能力，团队反而会因为缺少合适抽象而限制它们³⁴。这句话值得当成路线图看待。下一代协作工具需要把代理当成团队成员的一种特殊形态：它有任务状态、上下文订阅、权限边界、证据输出和交接协议。

Shared team state for agents

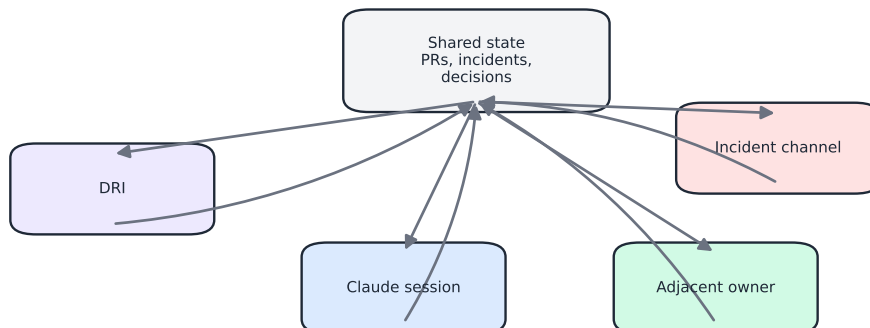


图 12: 多人协作的共享状态：DRI、Claude 会话、相邻责任区和事故频道都需要订阅同一组 PR、决策、风险和上下文更新。此图根据 ‘00:34:27–00:35:39’ 中关于 multi-player、chat sharing 和 teammate collaborator 抽象的讨论重绘。

³¹00:34:27–00:34:33。

³²00:34:33–00:35:20。

³³00:35:04–00:35:23。

³⁴00:35:27–00:35:39。

7.10 合并前问责：能上线之前，必须能解释

Dan 最后把问题落到“理解多少才够放心”。Mike 把答案拆成两个部分。第一是 verification：他已经非常重视验证，并把它类比为以前亲自写代码时追求 tightest dev loop 的习惯³⁵。在 Instagram 时代，一个紧 dev loop 可能是给某个屏幕单独做 Xcode build target，塞入 synthetic data，然后快速迭代³⁶。在代理时代，等价物变成：每个 Claude 产出的 PR 都附带照片或视频证据，尤其是 iOS PR 或 UI 改动³⁷。

这类证据会显著提升信心。比如 Fable 工作数小时后说完成，团队可以要求完整 screenshot gallery；人类可能在某张截图里看到一个从没亲眼遇到的错误状态，然后决定调整³⁸。这就是把“我相信模型说它完成了”转成“我看见关键路径和边界状态的证据”。

第二部分是 accountability。Mike 明确说，人在生产系统里仍要 stand behind the work³⁹。即便很多代码由 Claude 写出，工程师也需要理解主要决策、主要 trade-offs，以及让这些内容可理解所需的 artifacts⁴⁰。

合并前的责任标准

PR ready 距离 ready to merge 还差一层责任确认。合并前，DRI 必须能说明关键取舍、验证证据和未决风险。会议里有人问“did you do X or Y”，负责人如果答不出来，正确动作是暂停合并，查清楚后再继续^a。

^a00:38:21–00:38:35。

Mike 观察到，团队正在适应这种新规范：有人说 PR 已经准备好，旁人问了一个细节，负责人停顿后承认还不确定，并决定在合并前查明⁴¹。这听起来像效率下降，实际是代理时代的健康摩擦。代理让产出更快，团队必须保留能让责任落地的摩擦：追问、证据、解释、暂停和签字。

7.11 本章小结

本章的核心 claim 是：Anthropic 的团队协作已经从单人线性开发，转向 DRI 驱动的多代理并行工作模型。它的机制是：人类保留意图、上下文、风险和问责，多个 Claude 会话推进实现、审查、原型、验证和证据收集。对团队的实际含义是：代理越能做事，dashboard、生产意识、上下文转移、多人状态共享和合并前责任标准越要成为工程流程的一部分。

8 第七章：验证机制：让代理做完，也让人敢签字

本章回答一个很硬的问题：当 Claude Code 或 Fable 5 级别的代理已经能连续改代码、开 PR、甚至替人跟进反馈时，人凭什么敢签字？Mike 的答案集中在 **verification loop(验证闭环)**：让代理面对真实流程，留下可观察证据，再把修复、部署、客户确认串成完整回路。主持人在 00:38:39--00:38:48 追问截图、屏幕共享之外还有什么验证方式；Mike 的回答从真实账号、staging、回归路径、当前改动意图、视频捕获、mock 后端，一直讲到 Slack/MCP 反馈闭环与上线后的 follow-through。

³⁵00:36:12–00:36:31。

³⁶00:36:31–00:36:49。

³⁷00:36:53–00:37:07。

³⁸00:37:07–00:37:30。

³⁹00:37:39–00:37:45。

⁴⁰00:37:45–00:38:18。

⁴¹00:38:21–00:38:39。

本章核心判断

代理能“做完”只说明任务有产出；团队敢“签字”需要证据链。证据链至少包含三层：真实流程是否跑通、关键回归路径是否保持稳定、当前改动的意图是否被深度验证。对于界面和动画，还需要截图、屏幕共享或 video captures；对于复杂后端，还需要可靠 mock backend、in-memory substitutes 或可同步的 proxy。

这里的关键词是 **harness（执行与验证支架）**。它像试飞场：飞机可以在里面使用真实控制系统、真实仪表和近似真实的空气动力学条件，同时避开正式航线上的不可控风险。对代理开发而言，harness 包含 staging 账号、测试数据、理想 workflow 文本、截图/视频采集、mock 后端、PR 状态、部署状态和客户反馈线程。缺少这套支架，代理产出只能靠“看起来没问题”来判断；一旦进入生产系统，这种判断会迅速变成风险债务。

8.1 从真实流程开始：staging、真实账号与跳过无关 onboarding

Mike 先把验证的起点放在 *real flows*。他说，验证要尽量让系统运行真实流程，单纯把一段静态数据注入页面只覆盖局部片段；系统越复杂，静态注入越容易给出虚假的安心感(00:38:50--00:39:03)。他的例子很具体：Anthropic 投入了不少工作，让 iOS app 可以登录 staging，使用真实账号和真实数据（00:39:03--00:39:12）。

这里有一个容易被忽略的工程细节：真实账号验证需要跳过无关摩擦。Mike 明确提到，测试人员只想检查屏幕的第二部分时，不能每次都穿过 *eight-stage onboarding process*(00:39:12--00:39:19)。这个做法的目的很明确：把验证焦点对准当前风险。若每次测试都从新用户引导开始，测试时间会被重复流程吞掉，代理也更难稳定复现目标状态。

什么是 staging real account

staging 是生产环境的排练场：服务、权限、数据形状尽量贴近真实产品，外部影响受到控制。**real account** 指测试主体使用接近真实用户权限和真实数据结构的账号。它的价值在于暴露登录态、权限、缓存、同步、网络延迟、真实数据边界等问题；它的风险在于权限泄漏和测试数据污染。因此 shared secret、特殊入口、一次性测试账号、审计日志都应成为验证支架的一部分。

可以把它类比成医生做手术模拟：练习需要真实器械和接近真实的组织模型，同时不需要每次从挂号、缴费、排队开始。真实流程提供可信度，跳过无关 onboarding 提供效率。两者合起来，才有足够高的测试吞吐。

8.2 两条路径一起跑：well-known paths 与 current-change paths

Mike 接着提出一组关键区分：*well-known paths* 与 *the things you're exercising in the exact moment* (00:39:35--00:39:43)。前者是稳定、已知、反复执行的核心路径，主要价值是回归测试；后者是当前 PR 或当前任务真正想改变的行为，主要价值是确认“这次改动到底解决了什么”。

已知路径可以写成文本定义的理想 workflow。Mike 说，他们有些地方会把 *ideal workflows in text* 表达出来，让 Claude 反复检查(00:39:43--00:39:50)。这种做法很朴素，也很强：文本 workflow 像铁路时刻表，代理每次跑车都要对照它；一旦老路线断了，问题马上暴露。与此同时，Claude 还擅长表达当前改动的 intent，并对这个 intent 做深入验证（00:39:50--00:40:00）。

一个有用的直觉公式是：

$$V \approx \frac{N_{\text{evidence}}}{N_{\text{critical}}}$$

- V ：验证覆盖的直觉比例，表示关键流程中有多少已经拿到可观察证据。
- N_{critical} ：本次发布或修复涉及的关键流程数量，包括已知回归路径和当前改动路径。
- N_{evidence} ：已经通过真实流程、截图、视频、测试、mock 后端或客户确认拿到证据的关键流程数量。

这个公式只用于建立直觉，不能当成精密质量指标。它提醒团队：验证覆盖看的是证据覆盖；测试条数只提供局部信号。十个浅层断言可能只覆盖一个按钮；一个完整 video capture 加一次真实 staging 登录，可能覆盖登录态、数据加载、UI 状态、动画、错误提示和后端契约。

Verification harness

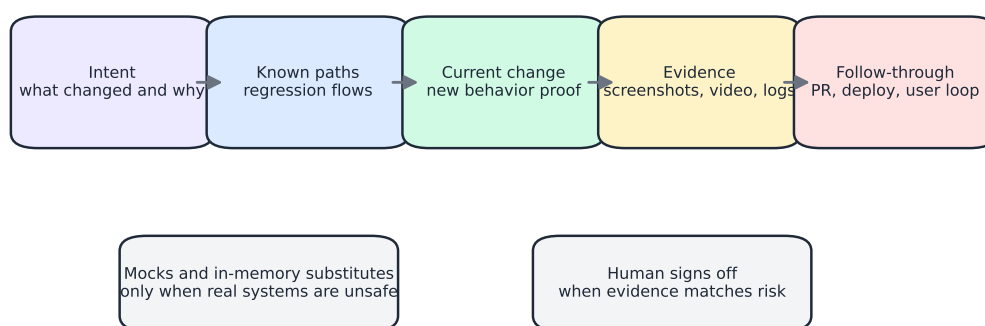


图 13: 验证支架：从变更意图出发，覆盖已知路径、当前变更证明、截图/视频/log 证据、PR/部署/用户反馈闭环；真实系统风险过高时才使用 mock 或内存替身。此图根据 ‘00:38:50–00:42:49’ 中关于真实流程、回归路径、视觉证据、视频验证、mock backend 和用户回访的讨论重绘。

这里的图是解释性重绘，依据来自字幕中的验证流程描述；本章涉及的真实视频画面主要用于确认访谈语境，而验证支架本身属于对原话机制的结构化抽象。

常见误区：把测试脚本当成验证闭环

测试脚本只是验证闭环的一种证据来源。闭环还要回答：测试环境是否像真实产品？路径是否覆盖老用户与新改动？界面是否可见地正确？复杂依赖是否被可靠替身覆盖？PR 合并后是否部署？反馈人是否确认问题已消失？这些问题缺一项，签字风险都会上升。

8.3 视觉证据：screenshots、screen shares 与 video captures

主持人先提到 screenshots 和 screen shares (00:38:43--00:38:48)，Mike 随后补充 video 的价值。他说 video 是给 Claude 的 *very underexplored tool* (00:40:02--00:40:09)。他的原型做法

是：把代理构建出来的东西录成 video captures，再给 Claude 观看和 scrub through，让它发现 “this animation has some jank in it” 并去修 (00:40:09--00:40:22)。

截图像一张体检报告，视频像一次运动监测。截图能看布局、颜色、内容是否出现；视频能看过渡、延迟、卡顿、滚动、焦点切换、动画节奏。Mike 特别指出，某些动画 jank 会被截图错过，因为截图只捕获一个瞬间 (00:40:22--00:40:30)。

为什么 video captures 对代理很重要

界面问题经常存在于时间维度：状态切换早了半秒、动画结束后元素跳动、加载骨架屏闪烁、滚动时焦点丢失、移动端键盘顶起布局。截图只能看到单帧状态；video captures 把时间序列交给代理分析。对 UI、动效和交互体验而言，这相当于给代理提供“过程证据”。

这也解释了为什么人工验收仍然重要。代理可以发现 jank，修复 jank，甚至解释修复理由；人仍要判断这个体验是否符合产品味道、品牌感和用户耐心。验证闭环可以降低盲区，无法替人承担产品判断。

8.4 复杂系统的替身：mock backend 与 in-memory substitutes

并非所有东西都容易端到端测试。Mike 提到，当系统复杂、端到端验证难以搭建时，可以让 Claude 构建尽可能 robust 的 *mock backend*，或者使用现成替身 (00:40:30--00:40:46)。这里的重点是“替身要足够像”，尤其要覆盖数据形状、错误状态、延迟、权限、连接中断和边界条件。

他回忆 Artifact 相关测试时提到，很多信息源都有很好的 *in-memory implementation*：Postgres、Redis、WebSocket 等依赖可以在单元测试中快速替换 (00:40:46--00:41:08)。换句话说，真实系统里的数据库、缓存和实时通道，在测试里可以用内存版实现出同样的契约。内存版不追求生产级性能，它追求快、稳定、可控、可复现。

mock backend、proxy 与 in-memory implementation

mock backend 是后端替身，负责模拟真实 API、数据和错误。**proxy** 在这段语境里接近 substitute：真实后端暂时难以启动，代理构建一个可替代的服务层让前端或 workflow 继续跑。**in-memory implementation** 是把数据库、缓存、队列、WebSocket 这类外部依赖放进内存里运行，常用于快速单元测试。三者的共同目标是降低验证启动成本，同时保留关键契约。

Mike 还讲了一个更有意思的变化：以前若有人提出“做一个替身后端，并随着上游变化保持同步”，他会担心上游一改，替身就漂移；现在他的判断变了，因为 Claude 可以读取变更、适配替身，并让两者保持同步 (00:41:28--00:41:44)。这里的实际含义是维护成本被代理压低了。团队仍需要契约测试、差异检查和审查，否则 mock 会变成漂亮的幻觉。

8.5 反馈闭环：Slack/MCP、PR、部署与客户跟进

验证闭环的后半段从“代码是否正确”扩展到“反馈是否真正关闭”。主持人在 00:41:44--00:41:58 提到一种架构：bug 出现后，代理自动启动、修复、关闭，并给客户发消息询问是否还看到问题。Mike 的回答把这个流程落到了 Slack 和 MCP 上：如果 bug report 来自 Slack feedback channel，通过 Slack MCP 可以拉取原始 thread，把上下文喂给 Claude Code，再让它回帖 (00:42:00--00:42:28)。

他给出的回帖口吻很有现场感 (00:42:28--00:42:31)：

Mike's Claude, I fixed it; here's the pull request.

更关键的是后续句子：“hold tight, it’s not in production yet; I’ll follow up when it actually is” (00:42:34--00:42:39)。这就是 follow-through: PR 只是中间节点，部署和客户确认才让反馈线程真正闭合。几小时后部署出去，代理再提示对方去测试，询问问题是否已修复 (00:42:40--00:42:49)。

身份代理与权限风险

Mike 也加了一个 disclaimer: 长时间运行的 Claude Code sessions 基本上是在 “as me” 地互动 (00:42:49--00:42:56)。这意味着团队必须处理身份、权限、审计和语气边界。代理可以代发状态更新，系统需要记录谁授权、发了什么、引用了哪些证据、是否越过了发布状态。

这里的 unknown unknown 是：很多团队会把 bug 修复看成代码问题，实际闭环跨越了协作系统。Slack thread、issue、PR、CI、部署、监控、客户确认是同一条链。链条中任何一段断掉，用户仍会感到 “没人负责”。Fable 5 的新价值，在 Mike 的描述里，正是更稳定地保持这条链不断。

8.6 修复判断：立即止血与长期治理要分层

最后一段讲的是 *discernment* (辨别力)。Mike 说，看到 bug report 后直接开修只是第一层能力；更高层能力是判断现在该止血、该修根因、该延后重构 (00:42:57--00:43:05)。他举了一个周末遇到的内部系统问题：系统长时间未重启，出现 memory leak。Claude 的判断是先 re-balance server，解决眼前问题，同时异步开 PR 做长期修复 (00:43:05--00:43:25)。

这段对工程团队很现实。SRE 或资深工程师遇到事故时，通常会把动作分成三层：

1. **止血**：先恢复服务，例如重启、扩容、切流、re-balance。
2. **根因修复**：定位 memory leak、资源泄漏、竞争条件或错误重试策略，并用 PR 修复。
3. **结构治理**：评估是否需要更换架构、语言、运行时或资源模型。

Mike 的重点是 balance: 像优秀 SRE 或 engineer in the loop 一样，先解决手头问题，再把 “大重构” 问题放到合适的决策窗口 (00:43:25--00:43:48)。代理如果只会 “接到 bug 就改代码”，会制造很多过度修复；代理如果能区分临时缓解、长期修复和架构治理，就开始具备生产协作价值。

生产验证的三问

每次让代理参与修复，都应逼它回答三问：第一，当前用户痛点是否已经被缓解？第二，长期 PR 是否解释了根因、证据和风险？第三，是否存在需要人类 DRI 决定的重构、权限、成本或产品体验问题？这三问把 “能修” 升级成 “能负责地推进”。

8.7 给团队的验证清单

把源视频第 09 段的验证机制压缩成团队清单，可以得到以下七项：

1. **真实流程**：能否在 staging 用真实账号、真实数据跑过目标路径？⁴²
2. **测试捷径**：是否有安全方式跳过无关新手引导，让测试直达目标状态？⁴³
3. **回归路径**：是否把已知路径写成可反复执行的理想工作流？⁴⁴

⁴²00:38:50--00:39:12。

⁴³00:39:12--00:39:32。

⁴⁴00:39:35--00:39:50。

4. **当前意图**：是否让代理明确本次改动 intent，并围绕它深度验证？⁴⁵
5. **视觉/视频证据**：截图是否足够？若问题存在于时间维度，是否补充视频捕获证据？⁴⁶
6. **复杂依赖替身**：替身后端、内存替身或代理是否覆盖关键契约，并能随上游同步？⁴⁷
7. **反馈跟进**：PR、部署、客户确认是否形成 follow-through？⁴⁸

这份清单的意义很直接：代理越强，验证清单越不能模糊。能力提升会扩大可委托范围，也会扩大误信范围。团队真正需要的是“代理执行 + 证据链 + 人类签字”的组合。

8.8 本章小结

本章的核心 claim 是：Fable 5 级别的代理让长程任务可委托，验证闭环让委托结果可签字。机制上，Mike 把真实 staging 流程、已知回归路径、当前改动意图、截图/视频证据、mock 后端、内存替身、Slack/MCP 反馈、PR、部署和客户确认连成一条证据链。实践含义很硬：团队要同时追问三件事：代理是否完成任务，证据覆盖了哪些关键流程，哪些风险仍由人类 DRI 承担。

9 第八章：还能建什么：领域专家、MCP 与复杂系统

这一章回答一个很实际的问题：当模型能力继续上升，普通人和专家各自多了什么可做的事？Dan 在上一段已经给出最简洁的框架：new models “raise the floor”，让更多人可以一次性搭出应用；同时也“raise the ceiling”，让软件工程师、创始人和领域专家完成过去很难靠个人推进的复杂项目（00:43:51–00:44:04）。这个判断要读得克制。它说的是可行工作空间变大，仍然要求人定义目标、给出领域知识、配置工具权限、审查结果和承担后果。

核心判断：地板和天花板同时上移

“Raised floor”指入门门槛下降：缺少传统工程背景的人，也能把熟悉领域里的需求变成可运行工具。“Raised ceiling”指上限抬高：已经懂系统、产品、数据或业务的人，可以把模型当作长程执行器，推进更大规模的架构迁移、仿真、可视化和内部系统集成（00:43:51–00:44:04）。

可以把这一变化想成一间工坊升级了工具。过去，一个领域专家可能只有纸笔、表格和少量自动化脚本；现在，专家可以把「这个流程为什么难」「哪些变量关键」「什么输出才算可靠」讲给模型，再让模型生成工具、仿真、可视化和验证支架。工具变强之后，真正稀缺的东西反而更清楚：领域判断、数据边界、权限治理和验收标准。

一个简单的工程地图可以写成：

$$B_{\text{domain}} \propto K_{\text{domain}} \times A_{\text{model}} \times T_{\text{tools}} \times V_{\text{verification}}$$

其中：

- B_{domain} ：某个领域问题被做成可用工具的可能性。
- K_{domain} ：领域专家掌握的问题结构、术语、约束和判断标准。

⁴⁵00:39:50--00:40:00。

⁴⁶00:40:02--00:40:30。

⁴⁷00:40:30--00:41:44。

⁴⁸00:42:00--00:42:49。

- A_{model} : 模型的规划、编码、调试、迁移和解释能力。
- T_{tools} : 工具接入，包括代码环境、业务系统、数据源、MCP、部署平台和权限。
- $V_{\text{verification}}$: 验证覆盖，包含真实流程、测试、截图、日志、人工复核和上线后反馈。

这个式子强调一件事：模型能力 A_{model} 很重要，单独看它会误导。领域专家知道哪里有坑，工具权限决定模型能触达什么系统，验证决定产物能否进入真实 workflow。

9.1 从 3D 游戏到复杂仿真：表达领域复杂性

Dan 先给出一个有趣例子：他说自己做了 Borges 的 “infinite library” 的一个版本，是一个在浏览器里运行的 3D game version，并且可以在里面找到许多文章 (00:44:07–00:44:19)。这里的教学重点放在表达方式变化：抽象内容可以变成可探索空间，文本资料可以变成互动界面，个人想法可以变成别人能进入和操作的東西。

Mike 顺着这个例子讲「fun side」：很多人有自己的世界、专业和复杂想法，现在可以思考如何把这种复杂性表达给别人，或者把其他领域的技术拿来解决自己的问题 (00:44:40–00:45:00)。他举了环境工程和地热研究的例子：复杂数学、仿真和跨领域技术可以被带入具体研究工作；自动字幕中近似出现 “PyTorch end-to-end simulations”，本文只把它作为 “复杂仿真能力” 这一类例子处理 (00:45:04–00:45:24)。

Raised floor and raised ceiling

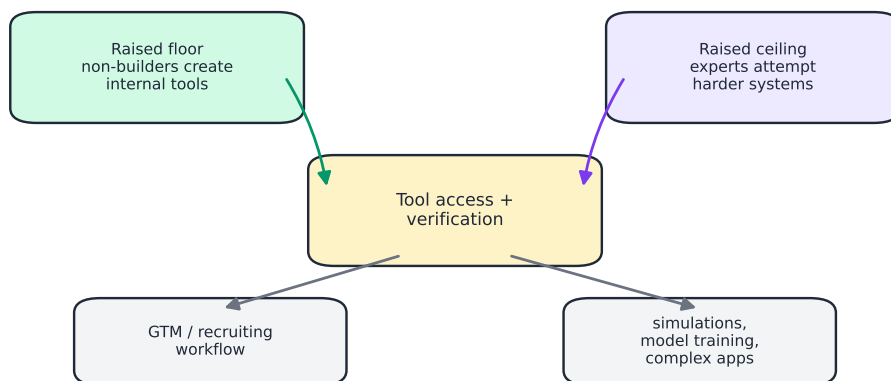


图 14: 能力边界同时向两端扩展：非传统 builder 更容易制造内部工具，专家 builder 可以尝试更复杂的仿真、训练和交互系统。此图根据 ‘00:44:07–00:45:39’ 中关于 3D infinite library、领域表达、仿真和跨领域技术迁移的讨论重绘。

领域专家为什么突然更能造工具

领域专家的优势来自「问题表征」。例如地热仿真的专家知道哪些变量、边界条件和异常输出有意义；GTM 人员知道客户、销售、支持和内部流程之间的断点。模型把自然语言、代码、接口和测试连接起来以后，这些专家可以直接参与工具生产，减少把所有细节先翻译成传统软件需求文档的等待成本 (00:45:04–00:46:55)。

这里有一个容易被低估的点：领域复杂性本身可以成为产品材料。过去，复杂性常常被压成会议纪要、流程图或「等工程团队排期」的需求。现在，复杂性可以被转成交互式解释器、仿真器、可视化仪表盘、数据整理工具，甚至是一个带有游戏结构的知识空间（00:44:07–00:45:36）。

9.2 MCP、权限和部署：内部系统才是真正的复杂系统

Mike 接着把话题从「有趣项目」推进到「组织内部工具」。他提到 Anthropic 正在努力让更多 internal systems 变成 MCP-friendly，并配上正确的 permissioning structure 和 deployment setup（00:45:45–00:46:00）。这句话很关键，因为内部系统的难点通常来自边界：哪些数据能读，哪些动作能写，谁批准，日志在哪里，失败后如何回滚。

未知风险：MCP 让能力进入组织边界

MCP 可以让模型接触 Slack、业务数据、内部服务或部署流程。能力进入组织边界后，权限、审计、最小授权、环境隔离和上线责任会立刻变成工程问题。随意接工具会扩大事故面；把权限、日志和回滚设计清楚，模型才适合进入关键流程（00:45:45–00:46:00）。

这也解释了 GTM 工具的意义。Mike 说有一位 go-to-market organization 的同事，把 Claude 深度整合进自己的整个流程，并且已经持续迭代数月，最后推进到整个 GTM organization 使用（00:46:06–00:46:44）。这个例子比一次性 demo 更有价值：它展示的是一个非传统技术出身的人，围绕自己熟悉的工作域，持续生长出越来越复杂的系统。

这里的关键词是“one shot”和持续迭代。早期模型常常适合做一次性小工具，复杂到一定程度后，继续改动会带来抽象混乱、破坏已有功能或维护压力。Mike 的说法是，Fable 或 Fable-like 能力让这个 GTM 系统继续增长，复杂度上限明显提高（00:46:18–00:46:55）。这仍然是视频中的经验性判断，正文把它限定为 Mike 的使用观察。

9.3 Senior engineer benchmark：有用，也要带着限定读

Dan 进一步给出自己的“senior engineer benchmark”：让模型从 first principles 重写一个代码库。字幕显示，之前最强模型大约是 62 或 63/100，这次模型得到 90 或 91，Dan 形容为 human senior engineer level（00:46:57–00:47:15）。

如何读 benchmark 分数

这一分数应写成「Dan 在视频中描述的内部/个人基准结果」，避免当成外部可复现的行业结论。精确表述应保留“90 or 91”和“62 or 63”的不确定性，因为字幕本身给出的就是近似口语表达（00:46:57–00:47:15）。

这个基准的教学价值在于任务类型：从 first principles 重写代码库，考验范围从单个函数补全扩展到系统理解、接口重建、行为保持、测试设计和迁移判断。它和后面的 dynamic workflows 正好接上：当模型能承担更长的系统级工作，人类需要把任务写成可运行流程，并把验证步骤放进流程本身。

9.4 本章小结

本章的核心 claim 是：Fable 5 这类模型同时降低入门门槛、抬高专家上限，因而扩大了「谁能造工具」和「能造多复杂工具」的范围。其机制是领域知识、模型能力、工具接入和验证闭环相乘，

而 MCP、权限和部署决定这些能力能否进入真实组织系统。实践含义很直接：领域专家可以开始把自己的工作域产品化，工程团队则需要把权限、审计、验证和责任边界提前设计好。

10 第九章：动态 workflow：把长程任务写成可运行的流程

如果第八章讨论「还能建什么」，第九章讨论「怎样让模型持续建下去」。Mike 说 dynamic workflows 是一个让他反复催 Sid 对外发布的内部工具；Sid 是构建这个能力的工程师，Mike 认为这个东西很适合 Fable 这样能做 deep meaningful work 的模型 (00:47:23–00:47:58)。这段话把前面所有主题收束到一个工程形态：长程任务需要支架，支架需要可检查的步骤。

定义：dynamic workflows

Dynamic workflows 指由人通过对话协商目标和步骤，再把流程表达成代码，最后用界面显示每个阶段执行状态的工作方式。它的价值在于把「一次聊天」扩展成「一组可运行、可审查、可追加的小流程」(00:50:08–00:51:05)。

Mike 给出的主例子是一个代码迁移：内部项目原本用 Python 写成，因为部署原因需要 TypeScript；最终得到的是 TypeScript and Bun port，而且有文档说明哪些部分没有迁移、原因是什么 (00:48:00–00:49:06)。过去在大型公司内部，把整个项目从一种技术栈迁移到另一种技术栈，通常意味着高昂协调成本、长期排期和大量人工复核。这里的变化是：人可以让 workflow 周末运行，回来后检查一个结构化迁移结果 (00:48:20–00:48:58)。

10.1 迁移案例：Python 到 TypeScript/Bun

这个 port 的关键超出「模型会翻译语法」。语法转换只是最浅的一层。真正的工作流包含至少六个阶段：

1. **Deep understanding**：先理解原项目的结构、行为、隐含约束和部署目的 (00:48:30–00:48:35)。
2. **Spec**：生成类似规格说明的文档，解释系统怎样工作 (00:48:35–00:48:38)。
3. **逐模块迁移**：按模块翻译并迁移，避免一次性生成一团新代码 (00:48:36–00:48:40)。
4. **Incremental tests**：每迁移一部分就测试一部分，降低错误定位成本 (00:48:39–00:48:41)。
5. **Adversarial test**：加入挑刺式测试，主动寻找失败模式 (00:48:41–00:48:43)。
6. **Missed-feature checks**：检查遗漏功能，并记录无法迁移或无需迁移的实现细节 (00:48:43–00:49:06)。

这六步共同构成一个迁移 harness。harness 的意思是执行与验证支架：它像登山绳索和保护点，让模型可以往前走，同时让人看到哪里可靠、哪里危险、哪里需要回退。

Dynamic workflow pipeline

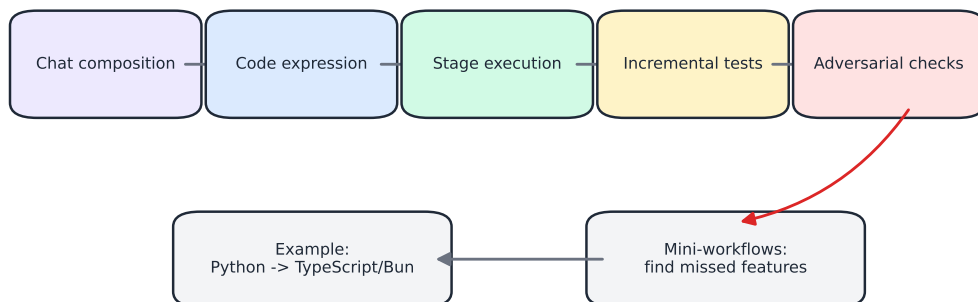


图 15: 动态工作流管线：先用 chat 组合任务，再把流程表达成代码，由系统分阶段执行，并通过增量测试、对抗性检查和后续 mini-workflow 修补遗漏。此图根据 ‘00:47:21–00:51:05’ 中 Python 到 TypeScript/Bun 迁移案例、workflow 设计和代码化执行的讨论重绘。

常见误读：把 workflow 当成大 prompt

Dynamic workflow 的重点在于流程对象本身。大 prompt 只是一段指令；workflow 还包含阶段边界、执行状态、测试点、失败处理、后续 mini workflows 和代码化表达。模型越能执行长任务，越需要这种支架来防止长程漂移（00:48:23–00:49:17）。

10.2 为什么代码化表达很重要

Dan 问 Mike 怎样设计这个 workflow，怎样确认它足够好。Mike 的答案很具体：他从 Claude Code 开始，说自己有一个复杂任务，希望一起设计 workflow；Claude Code 给出计划后，他继续要求加入三四层 additional verification for missed features（00:50:01–00:50:24）。这说明用户并没有把控制权交出去。他在定义目标、看计划、补充验收层级，并决定什么时候开始运行。

高信号原话片段：chat、code 与 UI（00:50:50–00:51:05）

Mike: “compose them using chat”，“expressed using code”，并且在执行时有一个干净的 UI 显示每个 stage 正在发生什么。

这段话是整集视频最适合作为工程原则保留下来的片段。Chat 适合协商意图，因为人可以用自然语言表达任务背景、偏好和限制；code 适合承载流程，因为它可审查、可复用、可版本化；UI 适合监督运行，因为长程任务需要让人看到阶段、状态、阻塞和结果。

进一步说，代码化 workflow 让团队获得三类资产：

- **可检查性：**人能在执行前看见模型准备进入哪些阶段，结束后再审查对应 diff。
- **可复用性：**一次成功的迁移流程可以变成同类任务的模板。

- **可组合性**：主流程完成后，后续疑问和小改动可以作为 mini workflows 继续追加 (00:50:31–00:50:41)。

10.3 模型选择也会进入工作流内部

Mike 还提到一个未来方向：动态工作流中的子任务可以按复杂度匹配不同 thinking level，某些步骤无需 extra high thinking，可以使用 medium thinking 或更小模型完成 (00:49:36–00:49:53)。这跟前文的模型选择章节互相呼应：模型选择可以从任务开始前延伸到 workflow 的每个阶段。

可以把一个动态工作流理解成：

$$W = \{(s_i, m_i, v_i)\}_{i=1}^n$$

其中：

- W ：完整工作流。
- s_i ：第 i 个 stage，例如理解、写 spec、迁移模块、测试、对抗检查。
- m_i ：该 stage 使用的模型或思考强度。
- v_i ：该 stage 的验证方式，例如单元测试、端到端检查、人工审查、日志或截图。
- n ：流程中的阶段数量。

这个表示法带来一个工程启发：高能力模型最适合放在高不确定性、高耦合、高判断压力的阶段；机械翻译、格式整理、简单测试修复等阶段可以交给较轻模型。这样做的目标是优化整个任务的成功率、时间和成本。

10.4 本章小结

本章的核心 claim 是：dynamic workflows 把长程代理任务从一次性聊天推进到可运行、可审查、可追加的流程。其机制是先理解系统、写出 spec、按模块迁移、增量测试、对抗测试、检查遗漏，再把后续问题变成 mini workflows。实践含义是：未来的工程能力会越来越像「设计流程和验证支架」，会写代码的人需要学会写可运行的工作流。

11 总结与延伸：新能力、新约束、新工程常识

这段结尾真正有信息量的部分停在 ‘00:51:05’ 左右：Mike 说 workflows 会把 long-horizon work 与 chat 连接起来 (00:50:50–00:51:05)。之后从 ‘00:51:17’ 开始进入音乐和订阅宣传，属于例行 outro，正文不纳入教学内容。

整集视频的主线可以压缩成一句话：Fable 5 的意义在于扩大可委托任务的时间尺度和复杂度，而新的工程常识随之转向意图表达、上下文管理、工具权限、验证支架和人类责任。第八、九章尤其说明，能力增长并不会自动变成可靠产物；能力要进入组织、代码库和真实用户场景，必须被流程化、可视化、可验证化。

11.1 新能力：把复杂想法做成可运行系统

第八章展示的新能力有三类。第一类是表达能力：把 infinite library 变成浏览器里的 3D 空间，把复杂领域知识变成游戏、可视化或仿真 (00:44:07–00:45:36)。第二类是领域工具能力：GTM、环境工程、研究仿真、内部流程都可以被领域专家直接塑形 (00:45:04–00:46:55)。第三类是系统迁移能力：一个复杂 Python 项目可以通过 workflow 迁移到 TypeScript/Bun，并留下迁移记录和验证步骤 (00:48:00–00:49:06)。

类比：从外包任务到内部生产线

普通 prompt 像把一个小任务外包出去；dynamic workflow 像搭一条临时生产线。生产线里有工序、质检点、负责人、异常处理和返工路径。模型能力越强，这条生产线越值得精心设计，因为它承载的工作更长、更复杂、更接近真实业务。

11.2 新约束：权限、验证、成本和责任

第八、九章也给出四个约束。第一，MCP 和内部系统接入带来权限风险，必须配置 permissioning、deployment setup 和审计机制 (00:45:45–00:46:00)。第二，benchmark 分数必须限定来源，Dan 的 senior engineer benchmark 是有价值的信号，同时仍是视频中的个人/内部基准描述 (00:46:57–00:47:15)。第三，长程任务的成本覆盖模型调用、review、rework、coordination 和 verification。第四，workflow 可以执行很多事，责任仍然落在人和团队身上。

工程常识的硬边界

模型能生成、迁移和测试代码，并不自动等于团队可以直接签字上线。真实生产系统需要权限最小化、审查日志、回滚方案、已知路径回归、当前变更验证和最终责任人。忽略这些边界，长程代理会把错误也执行得更远。

11.3 新工程常识：把工作写成流程

这一集给出的工程常识可以概括为三个动作：

1. **把目标讲清楚**：明确业务目标、领域约束、输入输出、成功标准和不可接受的失败。
2. **把流程写出来**：让模型先生成计划或 workflow，要求它把阶段、验证点和遗漏检查代码化表达 (00:50:08–00:50:31)。
3. **把证据留下来**：要求测试、日志、截图、文档、无法迁移清单和后续 mini workflows，方便人类复核和团队交接 (00:48:40–00:50:41)。

换句话说，未来的高水平使用者会同时像产品经理、架构师、测试工程师和编辑。他要能说清意图，也要能判断模型产物的形状；他要会拆任务，也要会设计验证；他要能让模型跑很久，也要知道什么时候暂停、审查和收束。

11.4 本章小结

- **能力侧**：Fable 5 的视频案例展示了地板和天花板同时上移，领域专家、GTM 人员、工程师和创始人都能推进更复杂的个人或组织工具 (00:43:51–00:46:55)。

- **机制侧**: dynamic workflows 通过 chat 组合、code 表达、UI 执行, 把长程任务拆成可运行阶段, 并把 deep understanding、spec、module translation、incremental tests、adversarial tests 和 missed-feature checks 放进流程 (00:47:23–00:51:05)。
- **风险侧**: benchmark、MCP、内部系统和自动迁移都需要来源限定、权限控制和验证闭环, 粗糙使用会把局部错误扩展成系统风险。
- **行动侧**: 下一步最值得练的是 workflow 设计能力: 先让模型提出流程, 再要求多层验证, 最后把执行结果、遗漏项和后续 mini workflows 作为交付物检查。